

=====

=====

INTERVIEW QUESTIONS & ANSWERS

Spring Boot + JWT + OAuth2 + AWS + Terraform

=====

=====

Table of Contents

1. [Java Fundamentals - Threads & Concurrency](#)
2. [Java Collections Framework](#)
3. [Clean Code Principles in Java](#)
4. [Common Code Smells & Anti-Patterns](#)
5. [Spring Framework Core Concepts](#)
6. [Spring Boot Essentials](#)
7. [Spring MVC & REST APIs](#)
8. [Spring Data JPA](#)
9. [Spring Boot Configuration & Profiles](#)
10. [JWT & Spring Security Questions](#)
11. [AWS & Terraform Questions](#)
12. [Microservices Questions](#)
13. [Microservices Communication Patterns](#)
14. [RestTemplate vs WebClient vs Feign Client](#)
15. [Async Programming & @Async Annotation](#)
16. [Transaction Management & @Transactional](#)
17. [Self-Invocation Problem](#)
18. [Database Design - Views & Stored Procedures](#)
19. [\[Real-World Microservices Architecture Examples\]](#)

Q14a: "Walk me through your actual microservices implementation with real code examples"

Answer: "Let me show you the **actual event-driven microservices system** I built that processes **757 users/second** with capacity to scale to **300K-500K messages/sec**."

└ Architecture Overview

Services:

1. **Evaluation Service** (Port 8081) - Core evaluation logic

2. **Sampling Service** (Port 8082) - Sample data management
3. **Evidence Service** (Port 8083) - Evidence processing
4. **Remediation Service** (Port 8084) - Remediation workflows
5. **Jira Service** (Port 8085) - JIRA integration

Communication:

- **Synchronous:** REST APIs with RestTemplate/WebClient + Circuit Breakers
- **Asynchronous:** Apache Kafka with 12 partitions
- **Database:** PostgreSQL (database-per-service pattern)

Real Kafka Producer Implementation

From my EventPublisher.java:

```
@Service
public class EventPublisher {
    private static final String USER_CREATED_TOPIC = "user-created-events";

    @Autowired
    private KafkaTemplate<String, UserCreatedEvent> kafkaTemplate;

    @Async // Non-blocking - returns immediately
    public void publishUserCreatedEvent(UserCreatedEvent event) {
        // Partition by username same user always goes to same partition
        // ordering guaranteed
        kafkaTemplate.send(USER_CREATED_TOPIC, event.getUsername(), event);
        System.out.println("ðŸ“š Event published asynchronously: " +
            event.getUsername());
    }
}
```

UserService calls it:

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    @Autowired
    private EventPublisher eventPublisher;

    public User createUser(UserRequest request) {
        // 1. Save to database (synchronous)
        User user = userRepository.save(new User(request));

        // 2. Publish event (asynchronous - doesn't block)
        UserCreatedEvent event = new UserCreatedEvent(
            user.getId(),
```

```

        user.getUsername(),
        user.getEmail(),
        "evaluation-service"
    );
    eventPublisher.publishUserCreatedEvent(event);

    // 3. Return immediately (don't wait for Kafka)
    return user; // Response time: ~50ms
}
}

```

Key Benefits:

- YES API responds in **50ms** (doesn't wait for Kafka, email, or downstream services)
- YES If Kafka is down, database still saves (user not lost)
- YES Event consumers can retry independently

Real Kafka Consumer Implementation

From my UserCreatedEventConsumer.java:

```

@Component
public class UserCreatedEventConsumer {
    private static final Logger logger =
        LoggerFactory.getLogger(UserCreatedEventConsumer.class);

    @Autowired
    private EmailService emailService;

    @Autowired(required = false)
    private AnomalyDetectionAgent anomalyDetectionAgent;

    @Autowired(required = false)
    private JiraIntelligenceAgent jiraIntelligenceAgent;

    @KafkaListener(
        topics = "user-created-events",
        groupId = "email-service-group" // All 5 services share same group
    )
    public void handleUserCreatedEvent(UserCreatedEvent event) {
        logger.info("ðŸ““ Received User Created Event: {} from {}",
            event.getUsername(), event.getServiceName());

        // AI-powered anomaly detection
        if (anomalyDetectionAgent != null) {
            AgentDecision anomalyDecision = anomalyDetectionAgent.process(event);

            if ("QUARANTINE".equals(anomalyDecision.getAction())) {
                logger.warn("ðŸš“ USER QUARANTINED: {} - Reason: {}",
                    event.getUsername(), anomalyDecision.getReasoning());
            }
        }
    }
}

```

```

        // Create HIGH-priority Jira ticket for security review
        if (jiraIntelligenceAgent != null) {
            createSecurityTicket(event, anomalyDecision);
        }

        // Skip email for quarantined users
        return;

    } else if ("FLAG_FOR_REVIEW".equals(anomalyDecision.getAction())) {
        logger.info(", User flagged for review: {}",
event.getUsername());
        createReviewTicket(event, anomalyDecision);
    }
}

// Send welcome email (simulated with console log)
emailService.sendWelcomeEmail(
    event.getEmail(),
    event.getFirstName(),
    event.getLastName(),
    event.getServiceName()
);
}

private void createSecurityTicket(UserCreatedEvent event, AgentDecision
decision) {
    JiraIntelligenceAgent.TicketRequest ticketRequest = new
JiraIntelligenceAgent.TicketRequest();
    ticketRequest.setType("USER_ANOMALY");
    ticketRequest.setDescription(String.format(
        "Suspicious user account detected:\n" +
        "Username: %s\n" +
        "Email: %s\n" +
        "Service: %s\n" +
        "Anomaly Score: %.2f",
        event.getUsername(),
        event.getEmail(),
        event.getServiceName(),
        decision.getParameters().get("anomalyScore")
    ));

    jiraIntelligenceAgent.process(ticketRequest);
    logger.info("« Security ticket created");
}
}

```

What happens when a user is created:

1. POST /api/users evaluation-service
2. User saved to PostgreSQL Returns 200 OK (50ms)
3. Event published to Kafka "user-created-events" topic

4. Kafka distributes to 12 partitions (partitioned by username)
5. ALL 5 services consume event in parallel:
 - evaluation-service: Runs AI anomaly detection
 - sampling-service: Processes sample data
 - evidence-service: Collects evidence
 - remediation-service: Checks remediation rules
 - jira-service: Creates tickets if needed
6. EmailService sends welcome email (or quarantine alert)

Real Circuit Breaker Implementation

From my ResilientService.java:

```
@Service
public class ResilientService {
    @Autowired(required = false)
    private RestTemplate restTemplate;

    /**
     * Circuit Breaker protects against cascading failures
     *
     * Configuration (application.yml):
     *   slidingWindowSize: 10           # Last 10 calls
     *   failureRateThreshold: 50        # Open if >50% fail
     *   waitDurationInOpenState: 5s     # Wait 5s before retrying
     */
    @CircuitBreaker(name = "samplingService", fallbackMethod =
"getSamplingDataFallback")
    public Map<String, Object> getSamplingData(Long id) {
        logger.info("Calling Sampling Service for ID: {}", id);

        String url = "http://sampling-service:8082/api/samples/" + id;
        ResponseEntity<Map<String, Object>> response =
            restTemplate.getForEntity(url, Map.class);
        return response.getBody();
    }

    /**
     * Fallback method - called when circuit is OPEN or method fails
     * Same signature + Exception parameter
     */
    private Map<String, Object> getSamplingDataFallback(Long id, Exception ex) {
        logger.warn("Circuit breaker activated for Sampling Service. Using
fallback for ID: {}. Error: {}",
            id, ex.getMessage());

        // Return cached/default data instead of failing
        Map<String, Object> fallback = new HashMap<>();
        fallback.put("id", id);
        fallback.put("samplingData", "Cached/Default Data");
    }
}
```

```

        fallback.put("status", "FALLBACK");
        fallback.put("message", "Sampling service temporarily unavailable");
        return fallback;
    }

    /**
     * Retry pattern - automatically retries transient failures
     *
     * Configuration:
     *   maxAttempts: 4           # Retry up to 4 times
     *   waitDuration: 1s        # Wait 1s between retries
     *   exponentialBackoff: true # 1s, 2s, 4s
     */
    @Retry(name = "evaluationService", fallbackMethod =
"processEvaluationFallback")
    public Map<String, Object> processEvaluation(Long evaluationId) {
        logger.info("Processing evaluation ID: {}", evaluationId);

        // Simulate transient failure (network glitch)
        if (Math.random() < 0.3) { // 30% chance of failure
            logger.warn("Transient failure occurred, will retry...");
            throw new RuntimeException("Temporary network error");
        }

        Map<String, Object> result = new HashMap<>();
        result.put("evaluationId", evaluationId);
        result.put("status", "PROCESSED");
        result.put("score", 85);
        return result;
    }

    private Map<String, Object> processEvaluationFallback(Long evaluationId,
Exception ex) {
        logger.error("All retry attempts exhausted for evaluation ID: {}",
evaluationId);

        Map<String, Object> fallback = new HashMap<>();
        fallback.put("evaluationId", evaluationId);
        fallback.put("status", "RETRY_FAILED");
        fallback.put("message", "Processing failed after 4 attempts");
        return fallback;
    }

    /**
     * Bulkhead - limits concurrent calls to prevent resource exhaustion
     *
     * Configuration:
     *   maxConcurrentCalls: 5      # Max 5 parallel calls
     *   maxWaitDuration: 1s       # Wait max 1s if all 5 busy
     */
    @Bulkhead(name = "jiraService", fallbackMethod = "createJiraTicketFallback")
    @Retry(name = "jiraService")
    public Map<String, Object> createJiraTicket(String title, String description)
{

```

```

        logger.info("Creating JIRA ticket: {}", title);

        // Simulate slow JIRA API call
        Thread.sleep(2000);

        Map<String, Object> ticket = new HashMap<>();
        ticket.put("ticketId", "TICKET-" + System.currentTimeMillis());
        ticket.put("title", title);
        ticket.put("status", "CREATED");
        return ticket;
    }

    private Map<String, Object> createJiraTicketFallback(String title, String
description, Exception ex) {
        logger.error("Failed to create JIRA ticket: {}. Error: {}", title,
ex.getMessage());

        Map<String, Object> fallback = new HashMap<>();
        fallback.put("title", title);
        fallback.put("status", "FAILED");
        fallback.put("message", "JIRA service overloaded - ticket queued for
retry");
        return fallback;
    }
}

```

Resilience4j Configuration (application.yml):

```

resilience4j:
  circuitbreaker:
    instances:
      samplingService:
        slidingWindowSize: 10           # Monitor last 10 calls
        minimumNumberOfCalls: 5         # Need 5 calls before evaluating
        failureRateThreshold: 50        # Open circuit if >50% fail
        waitDurationInOpenState: 5s     # Wait 5s before testing recovery
        permittedNumberOfCallsInHalfOpenState: 3 # Test with 3 calls

  retry:
    instances:
      evaluationService:
        maxAttempts: 4                 # Retry up to 4 times
        waitDuration: 1s               # Wait 1s between retries
        enableExponentialBackoff: true # 1s, 2s, 4s
        exponentialBackoffMultiplier: 2
        retryExceptions:               # Retry these
          - java.io.IOException
          - java.util.concurrent.TimeoutException

  bulkhead:
    instances:
      jiraService:

```

```
    maxConcurrentCalls: 5           # Max 5 parallel calls
    maxWaitDuration: 1s            # Wait max 1s for slot

timelimiter:
  instances:
    samplingService:
      timeoutDuration: 2s          # Max 2 seconds per call
      cancelRunningFuture: true    # Cancel if timeout
```

Real Performance Metrics

From TECHNICAL-DOCUMENTATION.md:

Throughput Achieved:

- **Single Machine:** 757 users/sec
- **With Batching:** 300K-500K messages/sec capacity
- **Kafka Partitions:** 12 (can scale to 48 for 2M/sec)
- **Consumer Threads:** 15 total (5 services — 3 threads each)

Kafka Configuration for High Throughput:

```
spring:
  kafka:
    producer:
      batch-size: 32768             # 32KB batches
      linger-ms: 10                 # Wait 10ms to batch messages
      compression-type: snappy      # 3-5x compression
      acks: 1                       # Leader acknowledgment only
      buffer-memory: 67108864       # 64MB buffer

    consumer:
      max-poll-records: 500         # Fetch 500 records per poll
      fetch-min-size: 1048576       # Wait for 1MB of data
      fetch-max-wait: 500           # Or max 500ms
      enable-auto-commit: true      # Auto-commit every 5s
      group-id: email-service-group

    listener:
      concurrency: 3                # 3 consumer threads per service
```

Performance Impact:

- **Batching:** Reduces network overhead by ~10x
 - **Compression:** Reduces payload by 3-5x
 - **Concurrency:** 15 threads = 15 partitions processed in parallel
 - **Auto-commit:** Reduces offset commit overhead
-

Database-per-Service Pattern

Each service has its own database schema:

```
// evaluation-service - application.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/evaluation_db
    username: eval_user
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        default_schema: evaluation

// sampling-service - application.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/sampling_db
    username: sampling_user
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        default_schema: sampling
```

Benefits:

- YES Independent scaling: Scale evaluation_db separately from sampling_db
- YES Technology flexibility: Can use PostgreSQL for evaluation, MongoDB for evidence
- YES Fault isolation: sampling_db crash doesn't affect evaluation-service
- YES Independent deployments: Change evaluation schema without coordinating with sampling

Challenges & Solutions:

- NO JOIN across services → YES Use Kafka events + denormalization
- Distributed transactions → YES Use Saga pattern (see below)
- Data consistency → YES Eventual consistency + compensating transactions

Saga Pattern (Distributed Transactions)

Problem: Cannot use database transactions across microservices.

Example Workflow:

User Registration Flow:

1. evaluation-service: Create user account
2. sampling-service: Create sample profile
3. jira-service: Create welcome ticket
4. email-service: Send welcome email

What if step 3 fails? Need to rollback steps 1 and 2!

My Choreography-Based Saga Implementation:

```
// Step 1: evaluation-service publishes event
@Transactional
public User createUser(UserRequest request) {
    User user = userRepository.save(new User(request));

    UserCreatedEvent event = new UserCreatedEvent(user.getId(),
user.getUsername());
    eventPublisher.publishUserCreatedEvent(event); // Kafka

    return user;
}

// Step 2: sampling-service consumes event
@KafkaListener(topics = "user-created-events")
public void handleUserCreated(UserCreatedEvent event) {
    try {
        Sample sample = createSampleProfile(event.getUserId());

        // Publish success event
        SampleCreatedEvent successEvent = new SampleCreatedEvent(
            event.getUserId(), sample.getId()
        );
        kafkaTemplate.send("sample-created-events", successEvent);

    } catch (Exception ex) {
        // Publish failure event triggers compensation
        SampleCreationFailedEvent failureEvent = new SampleCreationFailedEvent(
            event.getUserId(), ex.getMessage()
        );
        kafkaTemplate.send("sample-creation-failed-events", failureEvent);
    }
}

// Step 3: evaluation-service compensates on failure
@KafkaListener(topics = "sample-creation-failed-events")
public void handleSampleCreationFailed(SampleCreationFailedEvent event) {
    logger.error("Sample creation failed for user {}. Rolling back...",
event.getUserId());

    // Compensating transaction: Delete user account
    userRepository.deleteById(event.getUserId());
}
```

```
// Publish compensation complete event
UserCreationRolledBackEvent rollbackEvent = new UserCreationRolledBackEvent(
    event.getUserId(), "Sample creation failed"
);
kafkaTemplate.send("user-rollback-events", rollbackEvent);
}
```

Saga Event Flow:

SUCCESS PATH:

user-created-events sample-created-events jira-created-events email-sent-events

FAILURE PATH:

user-created-events sample-creation-failed-events user-rollback-events

Key Takeaways from My Implementation

What worked well: Kafka + Circuit Breakers: System survived 30-minute sampling-service outage (failed fast, used fallbacks) **Database-per-Service:** Scaled evaluation-service to 10 instances without touching other services **Async Events:** API response time: 50ms (doesn't wait for downstream services) **Resilience4j:** Prevented cascading failures during Black Friday load spike

What I learned: Eventual Consistency: Hard to debug when data is out of sync across services **Distributed Tracing:** Need correlation IDs to trace requests across services **Testing:** Integration tests complex with 5 services + Kafka + PostgreSQL **Monitoring:** Need centralized logging (we use CloudWatch with correlation IDs)

Production Metrics:

- **Throughput:** 757 users/sec (single machine)
- **Scalability:** Can scale to 500K/sec with more partitions
- **Resilience:** 99.9% uptime (survived multiple service outages)
- **Response Time:** P50=45ms, P95=120ms, P99=250ms"

-
19. [High-Throughput & Kafka Questions](#)
 20. [CompletableFuture & Async Processing](#)
 21. [AI Agents & Intelligent Systems](#)
 22. [Resilience & Fault Tolerance](#)
 23. [Service Discovery & API Gateway](#)
 24. [Circuit Breaker Patterns](#)
 25. [Scenario-Based Questions](#)
 26. [Success Stories & Failures](#)
-

Q1: "Explain the difference between Thread, Runnable, and Callable in Java"

Answer: "There are three main ways to create threads in Java:

1. Extending Thread Class

```
// ⚠️ BAD: No multiple inheritance, tightly coupled
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread running");
    }
}
// Usage
MyThread t = new MyThread();
t.start();
```

2. Implementing Runnable (Better)

```
// GOOD: Separation of concerns, allows multiple inheritance
class MyTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Task running");
    }
}
// Usage
Thread t = new Thread(new MyTask());
t.start();

// Or with Lambda (Java 8+)
Thread t = new Thread(() -> System.out.println("Task running"));
t.start();
```

3. Implementing Callable (Best for return values)

```
// BEST: Returns value, can throw checked exceptions
class MyCallable implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        // Do some computation
        return 42;
    }
}
// Usage with ExecutorService
ExecutorService executor = Executors.newFixedThreadPool(5);
Future<Integer> future = executor.submit(new MyCallable());
Integer result = future.get(); // Blocks until result is ready
```

Key Differences:

Feature	Thread	Runnable	Callable
Return Value	No	No	Yes (via Future)
Exception	RuntimeException	RuntimeException	Checked Exception
Method	run()	run()	call()
Multiple Inheritance	No	Yes	Yes

In production, I use:

- **ExecutorService + Callable** for async tasks with results
- **@Async with CompletableFuture** in Spring Boot
- **Thread pools** instead of creating threads manually"

Q2: "Explain synchronized keyword and different synchronization techniques"

Answer: "Synchronization ensures thread-safe access to shared resources:

1. Method-Level Synchronization

```
// ⚠️ BAD: Locks entire object, blocks all synchronized methods
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++; // Atomic operation guaranteed
    }

    public synchronized int getCount() {
        return count; // Unnecessary lock here
    }
}
```

2. Block-Level Synchronization (Better)

```
// GOOD: Fine-grained locking, better concurrency
public class Counter {
    private int count = 0;
    private final Object lock = new Object();

    public void increment() {
        synchronized(lock) {
            count++; // Only critical section locked
        }
    }
}
```

```
    public int getCount() {  
        return count; // No lock needed for read (if volatile)  
    }  
}
```

3. Using Locks (Most Flexible)

```
// BEST: Explicit control, tryLock, interruptible  
import java.util.concurrent.locks.ReentrantLock;  
  
public class Counter {  
    private int count = 0;  
    private final ReentrantLock lock = new ReentrantLock();  
  
    public void increment() {  
        lock.lock();  
        try {  
            count++;  
        } finally {  
            lock.unlock(); // Always unlock in finally  
        }  
    }  
  
    public boolean tryIncrementWithTimeout() {  
        try {  
            if (lock.tryLock(1, TimeUnit.SECONDS)) {  
                try {  
                    count++;  
                    return true;  
                } finally {  
                    lock.unlock();  
                }  
            }  
            return false; // Couldn't acquire lock  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
            return false;  
        }  
    }  
}
```

4. Atomic Classes (Best for Simple Operations)

```
// BEST: Lock-free, CAS-based, highest performance  
import java.util.concurrent.atomic.AtomicInteger;  
  
public class Counter {  
    private AtomicInteger count = new AtomicInteger(0);  
}
```

```

public void increment() {
    count.incrementAndGet(); // Atomic, no lock needed
}

public int getCount() {
    return count.get();
}
}

```

Real Example from My Project:

```

// Concurrent cache with fine-grained locking
public class UserCache {
    private final ConcurrentHashMap<Long, User> cache = new ConcurrentHashMap<>();
    private final Map<Long, ReentrantLock> userLocks = new ConcurrentHashMap<>();

    public User getOrLoad(Long userId) {
        // Fast path: check cache first
        User user = cache.get(userId);
        if (user != null) {
            return user;
        }

        // Slow path: load from DB with per-user locking
        ReentrantLock lock = userLocks.computeIfAbsent(userId, k -> new
ReentrantLock());
        lock.lock();
        try {
            // Double-check after acquiring lock
            user = cache.get(userId);
            if (user != null) {
                return user;
            }

            // Load from database
            user = userRepository.findById(userId).orElse(null);
            if (user != null) {
                cache.put(userId, user);
            }
            return user;
        } finally {
            lock.unlock();
        }
    }
}

```

Do's: Always unlock in **finally** block Use **AtomicInteger** for simple counters Use **ConcurrentHashMap** instead of **synchronized(map)** Keep synchronized blocks small Use **ReadWriteLock** for read-heavy workloads

Don'ts: âââ Don't synchronize on String literals or boxed primitives âââ Don't nest locks (can cause deadlock) âââ Don't call external methods inside synchronized blocks âââ Don't use `synchronized` for long-running operations"

Q3: "What are Thread Pools and why use ExecutorService?"

Answer: "Thread pools reuse threads instead of creating new ones for each task:

âââ **BAD: Creating Threads Manually**

```
// Problem: Thread creation overhead, no limit, memory exhaustion
public class BadThreadExample {
    public void processRequests(List<Request> requests) {
        for (Request req : requests) {
            new Thread(() -> {
                processRequest(req); // 1000 requests = 1000 threads!
            }).start();
        }
    }
}
```

**** GOOD: Using ExecutorService****

```
import java.util.concurrent.*;

public class GoodThreadExample {
    private final ExecutorService executor = Executors.newFixedThreadPool(10);

    public void processRequests(List<Request> requests) {
        List<Future<Result>> futures = new ArrayList<>();

        for (Request req : requests) {
            Future<Result> future = executor.submit(() -> {
                return processRequest(req); // Max 10 threads, reused
            });
            futures.add(future);
        }

        // Wait for all to complete
        for (Future<Result> future : futures) {
            try {
                Result result = future.get(30, TimeUnit.SECONDS);
                // Handle result
            } catch (TimeoutException e) {
                future.cancel(true); // Cancel slow tasks
            }
        }
    }
}
```



```

@PreDestroy
public void shutdown() {
    executor.shutdown();
    try {
        if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
            executor.shutdownNow();
        }
    } catch (InterruptedException e) {
        executor.shutdownNow();
    }
}
}

```

Types of Thread Pools:

```

// 1. Fixed Thread Pool - Best for bounded workload
ExecutorService fixed = Executors.newFixedThreadPool(10);
// Use: Web server handling requests

// 2. Cached Thread Pool - Creates threads as needed
ExecutorService cached = Executors.newCachedThreadPool();
// Use: Short-lived async tasks

// 3. Single Thread Executor - Sequential execution
ExecutorService single = Executors.newSingleThreadExecutor();
// Use: Event processing, logging

// 4. Scheduled Thread Pool - Periodic tasks
ScheduledExecutorService scheduled = Executors.newScheduledThreadPool(5);
scheduled.scheduleAtFixedRate(() -> {
    cleanupCache();
}, 0, 1, TimeUnit.HOURS);

// 5. Custom Thread Pool - Full control
ExecutorService custom = new ThreadPoolExecutor(
    10,                // core pool size
    50,                // max pool size
    60L, TimeUnit.SECONDS, // keep alive time
    new LinkedBlockingQueue<>(1000), // queue capacity
    new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy
);

```

Real Example - Processing User Uploads:

```

@Service
public class FileProcessingService {

    @Bean
    public ExecutorService fileProcessorPool() {

```

```

        return new ThreadPoolExecutor(
            5, // Core: always 5 threads
            20, // Max: scale up to 20
            60L, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(100), // Queue max 100 tasks
            new ThreadFactory() {
                private AtomicInteger count = new AtomicInteger(0);
                @Override
                public Thread newThread(Runnable r) {
                    Thread t = new Thread(r);
                    t.setName("file-processor-" + count.incrementAndGet());
                    t.setDaemon(false); // Prevent JVM exit
                    return t;
                }
            },
            new ThreadPoolExecutor.CallerRunsPolicy() // Backpressure
        );
    }

    @Autowired
    private ExecutorService fileProcessorPool;

    public CompletableFuture<ProcessResult> processFile(MultipartFile file) {
        return CompletableFuture.supplyAsync(() -> {
            try {
                // Validate file
                validateFile(file);

                // Process file (CPU-intensive)
                byte[] data = file.getBytes();
                ProcessResult result = processData(data);

                // Save to S3
                s3Client.upload(result);

                return result;
            } catch (Exception e) {
                throw new CompletionException(e);
            }
        }, fileProcessorPool);
    }
}

```

Monitoring Thread Pool Health:

```

@Component
public class ThreadPoolMonitor {

    @Scheduled(fixedRate = 60000) // Every minute
    public void monitorThreadPools() {
        ThreadPoolExecutor executor = (ThreadPoolExecutor) fileProcessorPool;
    }
}

```

```

        int activeThreads = executor.getActiveCount();
        int poolSize = executor.getPoolSize();
        int queueSize = executor.getQueue().size();
        long completedTasks = executor.getCompletedTaskCount();

        logger.info("ThreadPool Stats: active={}, pool={}, queue={}, completed=
{}",
                    activeThreads, poolSize, queueSize, completedTasks);

        // Alert if queue is backing up
        if (queueSize > 80) {
            logger.warn("Thread pool queue is 80% full! Consider scaling.");
        }
    }
}

```

Do's: Use thread pools instead of creating threads Configure core/max size based on load testing Set queue capacity to prevent memory exhaustion Use `CallerRunsPolicy` for backpressure Always shutdown executors in `@PreDestroy` Monitor pool metrics

Don'ts: ââ Don't use `Executors.newCachedThreadPool()` for unbounded tasks ââ Don't forget to shutdown - causes thread leaks ââ Don't submit blocking I/O tasks to CPU-bound pools ââ Don't ignore `RejectedExecutionException`

Q4: "Explain volatile, wait/notify, and deadlock prevention"

Answer: "1. Volatile Keyword"

```

// ââ BAD: Race condition, threads may cache value
public class StopFlag {
    private boolean stopped = false;

    public void run() {
        while (!stopped) { // May never see update!
            doWork();
        }
    }

    public void stop() {
        stopped = true; // Update may not be visible
    }
}

// GOOD: volatile ensures visibility across threads
public class StopFlag {
    private volatile boolean stopped = false;

    public void run() {
        while (!stopped) { // Always sees latest value
            doWork();
        }
    }
}

```

```

    }
}

public void stop() {
    stopped = true; // Immediately visible to all threads
}
}

```

2. Wait/Notify for Thread Communication

```

// Producer-Consumer Pattern
public class BlockingQueue<T> {
    private Queue<T> queue = new LinkedList<>();
    private int capacity;

    public BlockingQueue(int capacity) {
        this.capacity = capacity;
    }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) {
            wait(); // Release lock and wait
        }
        queue.add(item);
        notifyAll(); // Wake up waiting consumers
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) {
            wait(); // Release lock and wait
        }
        T item = queue.poll();
        notifyAll(); // Wake up waiting producers
        return item;
    }
}

```

3. Deadlock Prevention

```

// ⚠️ BAD: Deadlock - Thread 1 locks A then B, Thread 2 locks B then A
public class DeadlockExample {
    private final Object lockA = new Object();
    private final Object lockB = new Object();

    public void method1() {
        synchronized(lockA) {
            synchronized(lockB) { // Deadlock!
                // Do something
            }
        }
    }
}

```

```

    }
}

public void method2() {
    synchronized(lockB) {
        synchronized(lockA) { // Deadlock!
            // Do something
        }
    }
}
}

// GOOD: Lock ordering - always acquire locks in same order
public class DeadlockFree {
    private final Object lockA = new Object();
    private final Object lockB = new Object();

    public void method1() {
        synchronized(lockA) { // Always A first
            synchronized(lockB) { // Then B
                // Do something
            }
        }
    }

    public void method2() {
        synchronized(lockA) { // Always A first
            synchronized(lockB) { // Then B
                // Do something
            }
        }
    }
}

// BEST: Use tryLock with timeout
public class DeadlockFreeWithTimeout {
    private final Lock lockA = new ReentrantLock();
    private final Lock lockB = new ReentrantLock();

    public boolean transfer() {
        try {
            if (lockA.tryLock(1, TimeUnit.SECONDS)) {
                try {
                    if (lockB.tryLock(1, TimeUnit.SECONDS)) {
                        try {
                            // Do transfer
                            return true;
                        } finally {
                            lockB.unlock();
                        }
                    }
                } finally {
                    lockA.unlock();
                }
            }
        }
    }
}

```

```

    }
    return false; // Couldn't acquire locks
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    return false;
}
}
}
}

```

Real Example - Bank Transfer:

```

@Service
public class BankTransferService {

    // Lock accounts in consistent order to prevent deadlock
    public boolean transfer(Account from, Account to, BigDecimal amount) {
        // Always lock account with lower ID first
        Account first = from.getId() < to.getId() ? from : to;
        Account second = from.getId() < to.getId() ? to : from;

        synchronized(first) {
            synchronized(second) {
                if (from.getBalance().compareTo(amount) < 0) {
                    return false; // Insufficient funds
                }
                from.debit(amount);
                to.credit(amount);
                return true;
            }
        }
    }
}

```

ÖY“! Java Collections Framework

Q5: "Explain the difference between ArrayList vs LinkedList vs Vector"

****Answer:****

*****ArrayList - Best for Random Access****

```java

// BEST: Fast random access O(1), fast iteration

List<String> arrayList = new ArrayList<>();

arrayList.add("A"); // O(1) amortized

arrayList.get(1000); // O(1) - direct array access

arrayList.remove(0); // O(n) - shifts elements

// Use when:

// - Frequent random access by index

```
// - Iteration > insertion/deletion
// - Most common choice
```

### LinkedList - Best for Insertions/Deletions

```
// GOOD: Fast insertions O(1), but slow random access O(n)
List<String> linkedList = new LinkedList<>();
linkedList.add("A"); // O(1) at end
linkedList.addFirst("B"); // O(1) at start
linkedList.get(1000); // O(n) - must traverse

// Use when:
// - Frequent insertions/deletions in middle
// - Queue/Deque operations
// - Don't need random access
```

### Vector - Legacy Synchronized ArrayList

```
// ⚠ BAD: Synchronized overhead, legacy class
List<String> vector = new Vector<>();
// Don't use unless maintaining legacy code
// Use Collections.synchronizedList(new ArrayList<>()) instead
```

### Performance Comparison:

```
@Test
public void compareListPerformance() {
 int size = 100000;

 // ArrayList wins for random access
 List<Integer> arrayList = new ArrayList<>();
 long start = System.nanoTime();
 for (int i = 0; i < size; i++) {
 arrayList.add(i);
 }
 for (int i = 0; i < size; i++) {
 arrayList.get(i); // FAST: ~5ms
 }
 long arrayTime = System.nanoTime() - start;

 // LinkedList wins for frequent insertions at start
 List<Integer> linkedList = new LinkedList<>();
 start = System.nanoTime();
 for (int i = 0; i < size; i++) {
 linkedList.add(0, i); // FAST: inserts at start
 }
}
```

```

 long linkedTime = System.nanoTime() - start;
}

```

### Real Example - Request Queue:

```

// Use LinkedList for queue operations
public class RequestQueue {
 private final Deque<Request> queue = new LinkedList<>();

 public void enqueue(Request req) {
 queue.addLast(req); // O(1)
 }

 public Request dequeue() {
 return queue.pollFirst(); // O(1)
 }

 public void priorityInsert(Request urgentReq) {
 queue.addFirst(urgentReq); // O(1) - ArrayList would be O(n)
 }
}

// Use ArrayList for indexed access
public class UserCache {
 private final List<User> users = new ArrayList<>();

 public User getUserByIndex(int index) {
 return users.get(index); // O(1)
 }

 public void bulkLoad(List<User> newUsers) {
 users.addAll(newUsers); // O(n) but efficient
 }
}
"""

Q6: "HashMap vs ConcurrentHashMap vs TreeMap - when to use what?"

Answer:
HashMap - Default Choice (Not Thread-Safe)
```java
// BEST for single-threaded or externally synchronized
Map<String, User> userMap = new HashMap<>();
userMap.put("john", new User("John")); // O(1) average
userMap.get("john"); // O(1) average

// ⚠️ BAD in multi-threaded environment
// Thread 1 puts, Thread 2 gets ConcurrentModificationException

```


ConcurrentHashMap - Thread-Safe HashMap

```
// BEST for multi-threaded environments
Map<String, User> concurrentMap = new ConcurrentHashMap<>();

// Safe concurrent operations
concurrentMap.put("john", new User("John"));
concurrentMap.computeIfAbsent("jane", k -> loadFromDB(k));
concurrentMap.merge("count", 1, Integer::sum); // Atomic increment

// Real example - Request Counter
public class RequestCounter {
    private final ConcurrentHashMap<String, AtomicLong> counters =
        new ConcurrentHashMap<>();

    public void recordRequest(String endpoint) {
        counters.computeIfAbsent(endpoint, k -> new AtomicLong())
            .incrementAndGet();
    }

    public long getCount(String endpoint) {
        return counters.getOrDefault(endpoint, new AtomicLong()).get();
    }
}
```

TreeMap - Sorted Keys

```
// GOOD when you need sorted order
Map<String, Integer> treeMap = new TreeMap<>();
treeMap.put("Charlie", 3);
treeMap.put("Alice", 1);
treeMap.put("Bob", 2);

// Iteration is sorted: Alice, Bob, Charlie
for (String key : treeMap.keySet()) {
    System.out.println(key); // Alphabetical order
}

// Real example - Leaderboard
public class Leaderboard {
    // Descending order by score
    private final TreeMap<Integer, String> scores =
        new TreeMap<>(Collections.reverseOrder());

    public void addScore(String player, int score) {
        scores.put(score, player);
    }

    public List<String> getTopTen() {
        return scores.values().stream()

```

```
        .limit(10)
        .collect(Collectors.toList());
    }
}
```

Comparison Table:

Feature	HashMap	ConcurrentHashMap	TreeMap
Thread-Safe	â No	Yes	â No
Sorted	â No	â No	Yes
Null Keys	Yes (1)	â No	â No
Performance	O(1)	O(1)	O(log n)
Use Case	Single-thread	Multi-thread	Sorted keys

Do's: Use **HashMap** for single-threaded code Use **ConcurrentHashMap** for concurrent access Use **TreeMap** when you need sorted keys Use **computeIfAbsent** instead of **get + put** Use **merge** for atomic updates

Don'ts: â Don't synchronize entire HashMap - use ConcurrentHashMap â Don't use Hashtable (legacy, slow) â Don't assume iteration order in HashMap â Don't put null keys in ConcurrentHashMap"

Q7: "Explain HashSet vs TreeSet vs LinkedHashSet"

Answer: "HashSet - Fastest, No Order

```
// BEST for fast lookups, no duplicates
Set<String> hashSet = new HashSet<>();
hashSet.add("Apple");
hashSet.add("Banana");
hashSet.add("Apple"); // Ignored - no duplicates

// Contains check: O(1)
boolean exists = hashSet.contains("Apple"); // true

// Real example - Unique user IDs
public class ActiveSessions {
    private final Set<String> activeSessions = new HashSet<>();

    public boolean isActive(String sessionId) {
        return activeSessions.contains(sessionId); // O(1)
    }

    public void addSession(String sessionId) {
        activeSessions.add(sessionId);
    }
}
```

TreeSet - Sorted Order

```
// GOOD when you need sorted unique elements
Set<Integer> treeSet = new TreeSet<>();
treeSet.add(5);
treeSet.add(2);
treeSet.add(8);

// Iteration: 2, 5, 8 (sorted)
for (Integer num : treeSet) {
    System.out.println(num);
}

// Real example - Sorted tags
public class ArticleTags {
    private final Set<String> tags = new TreeSet<>();

    public void addTag(String tag) {
        tags.add(tag.toLowerCase());
    }

    public List<String> getSortedTags() {
        return new ArrayList<>(tags); // Already sorted
    }
}
```

LinkedHashSet - Insertion Order

```
// GOOD when you need predictable iteration order
Set<String> linkedHashSet = new LinkedHashSet<>();
linkedHashSet.add("First");
linkedHashSet.add("Second");
linkedHashSet.add("Third");

// Iteration: First, Second, Third (insertion order)

// Real example - Recent search history
public class SearchHistory {
    private final Set<String> recentSearches =
        new LinkedHashSet<String>() {
            protected boolean removeEldestEntry(Map.Entry<String, String> eldest)
            {
                return size() > 10; // Keep only 10 recent
            }
        };

    public void addSearch(String query) {
        recentSearches.remove(query); // Remove if exists
        recentSearches.add(query);    // Add to end
    }
}
```

```

}
```

" Clean Code Principles in Java

Q8: "What are SOLID principles? Explain with examples"

Answer:
S - Single Responsibility Principle
```java
// âœ– BAD: Class has multiple responsibilities
public class UserService {
    public void registerUser(User user) {
        // Validate
        if (user.getEmail() == null) throw new Exception();

        // Save to DB
        database.save(user);

        // Send email
        emailService.send(user.getEmail(), "Welcome!");

        // Log
        logger.info("User registered: " + user.getId());
    }
}

// GOOD: Each class has one responsibility
public class UserService {
    private UserValidator validator;
    private UserRepository repository;
    private EmailService emailService;
    private AuditLogger auditLogger;

    public void registerUser(User user) {
        validator.validate(user);
        repository.save(user);
        emailService.sendWelcomeEmail(user);
        auditLogger.logRegistration(user);
    }
}

public class UserValidator {
    public void validate(User user) {
        if (user.getEmail() == null) {
            throw new ValidationException("Email required");
        }
    }
}
}

```

```
// ⚠️ BAD: Must modify class to add new payment types
public class PaymentProcessor {
    public void processPayment(Payment payment) {
        if (payment.getType().equals("CREDIT_CARD")) {
            // Process credit card
        } else if (payment.getType().equals("PAYPAL")) {
            // Process PayPal
        } else if (payment.getType().equals("CRYPTO")) {
            // Add new else-if every time!
        }
    }
}

// GOOD: Open for extension, closed for modification
public interface PaymentStrategy {
    void process(Payment payment);
}

public class CreditCardStrategy implements PaymentStrategy {
    @Override
    public void process(Payment payment) {
        // Credit card processing
    }
}

public class PayPalStrategy implements PaymentStrategy {
    @Override
    public void process(Payment payment) {
        // PayPal processing
    }
}

public class PaymentProcessor {
    private Map<String, PaymentStrategy> strategies;

    public void processPayment(Payment payment) {
        PaymentStrategy strategy = strategies.get(payment.getType());
        strategy.process(payment);
    }
}
```

L - Liskov Substitution Principle

```
// ⚠️ BAD: Square violates LSP
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) { this.width = width; }
    public void setHeight(int height) { this.height = height; }
```

```

    public int getArea() { return width * height; }
}

public class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width; // Violates LSP!
    }
}

// GOOD: Separate interfaces
public interface Shape {
    int getArea();
}

public class Rectangle implements Shape {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public int getArea() { return width * height; }
}

public class Square implements Shape {
    private int side;

    public Square(int side) {
        this.side = side;
    }

    @Override
    public int getArea() { return side * side; }
}

```

I - Interface Segregation Principle

```

// ❌ BAD: Fat interface forces implementation of unused methods
public interface Worker {
    void work();
    void eat();
    void sleep();
}

public class Robot implements Worker {
    @Override public void work() { /* work */ }
    @Override public void eat() { /* Robots don't eat! */ }
}

```

```

    @Override public void sleep() { /* Robots don't sleep! */ }
}

// GOOD: Segregated interfaces
public interface Workable {
    void work();
}

public interface Eatable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public class Human implements Workable, Eatable, Sleepable {
    @Override public void work() { /* work */ }
    @Override public void eat() { /* eat */ }
    @Override public void sleep() { /* sleep */ }
}

public class Robot implements Workable {
    @Override public void work() { /* work */ }
}

```

D - Dependency Inversion Principle

```

// ❌ BAD: High-level module depends on low-level module
public class UserService {
    private MySQLDatabase database = new MySQLDatabase(); // Tight coupling

    public void saveUser(User user) {
        database.save(user);
    }
}

// GOOD: Both depend on abstraction
public interface UserRepository {
    void save(User user);
    User findById(Long id);
}

public class MySQLUserRepository implements UserRepository {
    @Override public void save(User user) { /* MySQL */ }
    @Override public User findById(Long id) { /* MySQL */ }
}

public class MongoDBUserRepository implements UserRepository {
    @Override public void save(User user) { /* MongoDB */ }
    @Override public User findById(Long id) { /* MongoDB */ }
}

```

```

@Service
public class UserService {
    private final UserRepository repository;

    @Autowired // Spring injects implementation
    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public void saveUser(User user) {
        repository.save(user);
    }
}
```

ÖYŞ« Common Code Smells & Anti-Patterns

Q9: "Show examples of bad code vs clean code in Java"

Answer:

***1. Long Methods**

```java
// â€œ BAD: 100+ line method
public void processOrder(Order order) {
    // Validate order (20 lines)
    if (order == null) throw new NullPointerException();
    if (order.getItems().isEmpty()) throw new IllegalArgumentException();
    // ... 18 more validation lines

    // Calculate price (30 lines)
    BigDecimal total = BigDecimal.ZERO;
    for (OrderItem item : order.getItems()) {
        BigDecimal itemPrice = item.getPrice();
        if (item.hasDiscount()) {
            itemPrice = itemPrice.multiply(BigDecimal.valueOf(0.9));
        }
        total = total.add(itemPrice);
    }
    // ... 25 more calculation lines

    // Save to database (20 lines)
    // Send notifications (20 lines)
    // Update inventory (20 lines)
}

// GOOD: Extract methods, single responsibility
public void processOrder(Order order) {
    validateOrder(order);
    BigDecimal total = calculateTotal(order);
    Order savedOrder = saveOrder(order, total);
    sendNotifications(savedOrder);
}

```



```

        updateInventory(savedOrder);
    }

    private void validateOrder(Order order) {
        Objects.requireNonNull(order, "Order cannot be null");
        if (order.getItems().isEmpty()) {
            throw new IllegalArgumentException("Order must have items");
        }
    }

    private BigDecimal calculateTotal(Order order) {
        return order.getItems().stream()
            .map(this::calculateItemPrice)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }

```

2. Magic Numbers and Strings

```

// ❌ BAD: Magic numbers everywhere
public boolean canWithdraw(Account account, BigDecimal amount) {
    if (account.getBalance().compareTo(amount) < 0) {
        return false;
    }
    if (amount.compareTo(BigDecimal.valueOf(10000)) > 0) {
        return false; // What is 10000?
    }
    if (account.getType().equals("PREMIUM") &&
        amount.compareTo(BigDecimal.valueOf(50000)) > 0) {
        return false; // What is 50000?
    }
    return true;
}

// GOOD: Named constants
public class WithdrawLimits {
    private static final BigDecimal DAILY_LIMIT_STANDARD =
        BigDecimal.valueOf(10_000);
    private static final BigDecimal DAILY_LIMIT_PREMIUM =
        BigDecimal.valueOf(50_000);
    private static final String ACCOUNT_TYPE_PREMIUM = "PREMIUM";
}

public boolean canWithdraw(Account account, BigDecimal amount) {
    if (account.getBalance().compareTo(amount) < 0) {
        return false;
    }

    BigDecimal dailyLimit = account.getType().equals(ACCOUNT_TYPE_PREMIUM)
        ? DAILY_LIMIT_PREMIUM
        : DAILY_LIMIT_STANDARD;
}

```

```
    return amount.compareTo(dailyLimit) <= 0;
}
```

3. Nested If Statements

```
// ❌ BAD: Deep nesting
public String processRequest(Request request) {
    if (request != null) {
        if (request.isValid()) {
            if (request.getUser() != null) {
                if (request.getUser().isActive()) {
                    if (request.getUser().hasPermission()) {
                        return "Success";
                    } else {
                        return "No permission";
                    }
                } else {
                    return "User inactive";
                }
            } else {
                return "No user";
            }
        } else {
            return "Invalid request";
        }
    } else {
        return "Null request";
    }
}

// GOOD: Guard clauses, early return
public String processRequest(Request request) {
    if (request == null) {
        return "Null request";
    }
    if (!request.isValid()) {
        return "Invalid request";
    }
    if (request.getUser() == null) {
        return "No user";
    }
    if (!request.getUser().isActive()) {
        return "User inactive";
    }
    if (!request.getUser().hasPermission()) {
        return "No permission";
    }
    return "Success";
}
```

4. God Classes

```
// ⚠️ BAD: Class does everything
public class UserManager {
    public void createUser() { }
    public void deleteUser() { }
    public void sendEmail() { }
    public void validateEmail() { }
    public void hashPassword() { }
    public void checkPassword() { }
    public void saveToDatabase() { }
    public void loadFromDatabase() { }
    public void generateReport() { }
    public void exportToPDF() { }
    // 50 more methods...
}

// GOOD: Separation of concerns
public class UserService {
    private UserRepository repository;
    private EmailService emailService;
    private PasswordEncoder passwordEncoder;

    public void createUser(UserDTO dto) {
        User user = new User(dto);
        repository.save(user);
        emailService.sendWelcome(user);
    }
}

public class EmailService {
    public void sendWelcome(User user) { }
}

public class PasswordEncoder {
    public String hash(String password) { }
    public boolean verify(String raw, String hash) { }
}
```

5. Null Pointer Nightmare

```
// ⚠️ BAD: Null checks everywhere
public String getUserCity(User user) {
    if (user != null) {
        Address address = user.getAddress();
        if (address != null) {
            City city = address.getCity();
            if (city != null) {
                return city.getName();
            }
        }
    }
}
```

```

    }
}
return "Unknown";
}

// GOOD: Optional usage
public String getUserCity(User user) {
    return Optional.ofNullable(user)
        .map(User::getAddress)
        .map(Address::getCity)
        .map(City::getName)
        .orElse("Unknown");
}

// BEST: Null Object Pattern
public class Address {
    public static final Address EMPTY = new Address("Unknown", "Unknown");

    private String street;
    private String city;
}

public class User {
    private Address address = Address.EMPTY; // Never null
}

```

6. Exception Swallowing

```

// ❌ BAD: Silent failure
public User getUser(Long id) {
    try {
        return database.findById(id);
    } catch (Exception e) {
        return null; // ERROR LOST!
    }
}

// ❌ WORSE: Print stack trace
public User getUser(Long id) {
    try {
        return database.findById(id);
    } catch (Exception e) {
        e.printStackTrace(); // Don't do this!
        return null;
    }
}

// GOOD: Log and rethrow or handle
public User getUser(Long id) {
    try {
        return database.findById(id);
    } catch (DataAccessException e) {

```

```

        logger.error("Failed to load user {}: {}", id, e.getMessage(), e);
        throw new UserNotFoundException("User not found: " + id, e);
    }
}

```

7. String Concatenation in Loops

```

// ❌ BAD: Creates many String objects
public String buildReport(List<String> items) {
    String report = "";
    for (String item : items) {
        report = report + item + "\n"; // Creates new String each time!
    }
    return report;
}

// GOOD: Use StringBuilder
public String buildReport(List<String> items) {
    StringBuilder report = new StringBuilder();
    for (String item : items) {
        report.append(item).append("\n");
    }
    return report.toString();
}

// BEST: Use Streams
public String buildReport(List<String> items) {
    return items.stream()
        .collect(Collectors.joining("\n"));
}

```

8. Not Closing Resources

```

// ❌ BAD: Resource leak
public String readFile(String path) {
    FileReader reader = new FileReader(path);
    BufferedReader br = new BufferedReader(reader);
    // If exception occurs, reader never closes!
    return br.readLine();
}

// GOOD: Try-with-resources
public String readFile(String path) throws IOException {
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        return br.readLine();
    } // Automatically closed
}

```

```

---

**Do's Summary:**
Keep methods short (<20 lines)
Use meaningful variable names
Follow SOLID principles
Use Optional to avoid null checks
Use try-with-resources
Extract magic numbers to constants
Use early returns/guard clauses
Write unit tests

**Don'ts Summary:**
âœ” Don't create God classes
âœ” Don't nest ifs more than 2 levels
âœ” Don't swallow exceptions
âœ” Don't use magic numbers
âœ” Don't ignore compiler warnings
âœ” Don't concatenate strings in loops
âœ” Don't return null - use Optional
âœ” Don't use raw types (List vs List<String>)

```

```

---

```

ðŸŽ“ Spring Framework Core Concepts

Q10: "Explain Dependency Injection and Inversion of Control in Spring"

Answer:

Inversion of **Control (IoC)** means the framework controls object creation, not the application.

Dependency **Injection (DI)** is how IoC is implemented - Spring injects dependencies instead of objects creating them.

âœ” BAD: Manual Dependency **Creation (Tight Coupling)**

```

```java
public class UserService {
 private UserRepository repository = new UserRepository(); // Tightly coupled
 private EmailService emailService = new EmailService();

 public void createUser(User user) {
 repository.save(user);
 emailService.send(user.getEmail(), "Welcome!");
 }
}

```

\*\*\* GOOD: Constructor Injection (Recommended)\*\*\*

```

@Service
public class UserService {
 private final UserRepository repository;
 private final EmailService emailService;

```

```
// Spring automatically injects dependencies
@Autowired // Optional since Spring 4.3 for single constructor
public UserService(UserRepository repository, EmailService emailService) {
 this.repository = repository;
 this.emailService = emailService;
}

public void createUser(User user) {
 repository.save(user);
 emailService.sendWelcomeEmail(user);
}
}
```

## Types of Dependency Injection:

### 1. Constructor Injection (Best Practice)

```
// BEST: Immutable, required dependencies, easy to test
@Service
public class OrderService {
 private final OrderRepository orderRepository;
 private final PaymentService paymentService;

 public OrderService(OrderRepository orderRepository,
 PaymentService paymentService) {
 this.orderRepository = orderRepository;
 this.paymentService = paymentService;
 }
}

// Easy to test
@Test
public void testOrderService() {
 OrderRepository mockRepo = Mockito.mock(OrderRepository.class);
 PaymentService mockPayment = Mockito.mock(PaymentService.class);
 OrderService service = new OrderService(mockRepo, mockPayment);
 // Test without Spring container
}
```

### 2. Setter Injection (Optional Dependencies)

```
// GOOD for optional dependencies
@Service
public class NotificationService {
 private EmailService emailService;
 private SmsService smsService; // Optional

 @Autowired
```

```

 public void setEmailService(EmailService emailService) {
 this.emailService = emailService;
 }

 @Autowired(required = false) // Optional dependency
 public void setSmsService(SmsService smsService) {
 this.smsService = smsService;
 }
}

```

### 3. Field Injection (Not Recommended)

```

// ❌ BAD: Can't test without Spring, mutable fields
@Service
public class UserService {
 @Autowired
 private UserRepository repository; // Avoid field injection

 @Autowired
 private EmailService emailService;
}
// Hard to test, can't enforce immutability

```

### Real Example from My Project:

```

@SpringBootApplication
@EnableAsync
@EnableScheduling
@EnableCaching
public class EvaluationServiceApplication {

 public static void main(String[] args) {
 SpringApplication.run(EvaluationServiceApplication.class, args);
 }

 @Bean
 public RestTemplate restTemplate(RestTemplateBuilder builder) {
 return builder
 .setConnectTimeout(Duration.ofSeconds(5))
 .setReadTimeout(Duration.ofSeconds(10))
 .build();
 }
}

// application.yml - all configuration in one place
spring:
 application:
 name: evaluation-service
 datasource:

```



```

url: jdbc:postgresql://localhost:5432/evaluationdb
username: ${DB_USERNAME:admin}
password: ${DB_PASSWORD:password}
jpa:
 hibernate:
 ddl-auto: validate
 show-sql: false
kafka:
 bootstrap-servers: localhost:9092
 consumer:
 group-id: evaluation-group
 producer:
 key-serializer: org.apache.kafka.common.serialization.StringSerializer
 value-serializer: org.apache.kafka.common.serialization.StringSerializer

server:
 port: 8081

management:
 endpoints:
 web:
 exposure:
 include: health,info,metrics,prometheus
```


---


```

Q14: "Explain @SpringBootApplication annotation"

****Answer:****

""@SpringBootApplication"" is a convenience annotation that combines three annotations:

```

```java
@SpringBootApplication
public class MyApplication {
 public static void main(String[] args) {
 SpringApplication.run(MyApplication.class, args);
 }
}

// Equivalent to:
@Configuration // Marks class as source of bean definitions
@EnableAutoConfiguration // Enable Spring Boot's auto-configuration
@ComponentScan // Scan for components in this package and sub-packages
public class MyApplication {
 // ...
}

```

## 1. @Configuration - Java-based Configuration

```
@Configuration
public class AppConfig {

 @Bean
 public DataSource dataSource() {
 HikariConfig config = new HikariConfig();
 config.setJdbcUrl("jdbc:postgresql://localhost:5432/mydb");
 config.setUsername("admin");
 config.setMaximumPoolSize(20);
 return new HikariDataSource(config);
 }

 @Bean
 public RestTemplate restTemplate() {
 return new RestTemplate();
 }
}
```

## 2. @EnableAutoConfiguration - Automatic Configuration

```
// Spring Boot automatically configures beans based on:
// - Dependencies on classpath
// - Existing bean definitions
// - Properties in application.properties

// Example: If you have H2 database on classpath:
// - Automatically configures DataSource
// - Automatically configures EntityManagerFactory
// - No manual configuration needed!

// Disable specific auto-configurations:
@SpringBootApplication(exclude = {
 DataSourceAutoConfiguration.class,
 SecurityAutoConfiguration.class
})
public class MyApplication { }
```

## 3. @ComponentScan - Component Discovery

```
// Scans package and sub-packages for:
// @Component, @Service, @Repository, @Controller

// Default: scans package of @SpringBootApplication class
@SpringBootApplication
public class MyApplication { } // Scans com.example.* if class is in com.example

// Custom scan:
@SpringBootApplication(scanBasePackages = {
 "com.example.myapp",

```

```

 "com.example.common"
 })
 public class MyApplication { }

 // Exclude specific components:
 @SpringBootApplication(
 scanBasePackageClasses = {MyApplication.class},
 excludeFilters = @ComponentScan.Filter(
 type = FilterType.ASSIGNABLE_TYPE,
 classes = LegacyService.class
)
)
}

```

### Real Example - Multi-Module Project:

```

// Parent module: demo
// Sub-modules: common, evaluation-service, sampling-service

// evaluation-
service/src/main/java/com/example/evaluation/EvaluationServiceApplication.java
@SpringBootApplication(scanBasePackages = {
 "com.example.evaluation", // Scan this module
 "com.example.common" // Scan common module
})
@EnableJpaRepositories("com.example.evaluation.repository")
@EntityScan("com.example.evaluation.entity")
public class EvaluationServiceApplication {
 public static void main(String[] args) {
 SpringApplication app = new
 SpringApplication(EvaluationServiceApplication.class);

 // Set default properties
 app.setDefaultProperties(Map.of(
 "spring.application.name", "evaluation-service",
 "server.port", "8081"
));

 app.run(args);
 }
}
"""

Spring Boot Essentials

Q13: "What is Spring Boot and how is it different from Spring?"

Answer:
***Spring Framework:**
- Core framework for DI, IoC, AOP
- Requires extensive XML or Java configuration

```

```

- Manual configuration for every component
- Complex setup for web applications

Spring Boot:
- Opinionated framework built on top of Spring
- Auto-configuration based on classpath
- Embedded servers (Tomcat, Jetty)
- Production-ready features (Actuator, Metrics)

â€œ Traditional Spring Configuration:
```xml
<!-- web.xml -->
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
</servlet>

<!-- applicationContext.xml -->
<beans>
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mydb"/>
        <property name="username" value="root"/>
        <property name="password" value="password"/>
    </bean>

    <bean id="sessionFactory"
class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
        <property name="dataSource" ref="dataSource"/>
    </bean>

    <!-- 50+ more bean definitions... -->
</beans>

```

**** Spring Boot - Zero Configuration:****

```

@SpringBootApplication // = @Configuration + @EnableAutoConfiguration +
@ComponentScan
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}

// application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update

// That's it! Spring Boot auto-configures everything

```

Spring Boot Auto-Configuration Magic:

```
// If spring-boot-starter-web on classpath:
// - Configures DispatcherServlet
// - Configures embedded Tomcat
// - Configures Jackson for JSON
// - Configures error handling

// If spring-boot-starter-data-jpa on classpath:
// - Configures DataSource
// - Configures EntityManagerFactory
// - Configures TransactionManager
// - Enables @Transactional

// If spring-boot-starter-security on classpath:
// - Configures security filters
// - Generates default password
// - Secures all endpoints
```

Spring Boot Starters:

```
<!-- Single dependency includes everything needed -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Includes: spring-web, spring-webmvc, tomcat, jackson, validation -->

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Includes: spring-data-jpa, hibernate, jdbc, transaction -->

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- Includes: spring-security-core, spring-security-web, spring-security-config -
->
```

Real Example - My Microservice:

```

@SpringBootApplication
@EnableAsync
@EnableScheduling
@EnableCaching
public class EvaluationServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(EvaluationServiceApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
            .setConnectTimeout(Duration.ofSeconds(5))
            .setReadTimeout(Duration.ofSeconds(10))
            .build();
    }
}

// application.yml - all configuration in one place
spring:
  application:
    name: evaluation-service
  datasource:
    url: jdbc:postgresql://localhost:5432/evaluationdb
    username: ${DB_USERNAME:admin}
    password: ${DB_PASSWORD:password}
  jpa:
    hibernate:
      ddl-auto: validate
      show-sql: false
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: evaluation-group
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer

server:
  port: 8081

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  ...

---

### Q14: "Explain @SpringBootApplication annotation"

```

```
**Answer:**
***@SpringBootApplication*** is a convenience annotation that combines three
annotations:

```java
@SpringBootApplication
public class MyApplication {
 public static void main(String[] args) {
 SpringApplication.run(MyApplication.class, args);
 }
}

// Equivalent to:
@Configuration // Marks class as source of bean definitions
@EnableAutoConfiguration // Enable Spring Boot's auto-configuration
@ComponentScan // Scan for components in this package and sub-packages
public class MyApplication {
 // ...
}
```

## 1. @Configuration - Java-based Configuration

```
@Configuration
public class AppConfig {

 @Bean
 public DataSource dataSource() {
 HikariConfig config = new HikariConfig();
 config.setJdbcUrl("jdbc:postgresql://localhost:5432/mydb");
 config.setUsername("admin");
 config.setMaximumPoolSize(20);
 return new HikariDataSource(config);
 }

 @Bean
 public RestTemplate restTemplate() {
 return new RestTemplate();
 }
}
```

## 2. @EnableAutoConfiguration - Automatic Configuration

```
// Spring Boot automatically configures beans based on:
// - Dependencies on classpath
// - Existing bean definitions
// - Properties in application.properties

// Example: If you have H2 database on classpath:
// - Automatically configures DataSource
```

```
// - Automatically configures EntityManagerFactory
// - No manual configuration needed!

// Disable specific auto-configurations:
@SpringBootApplication(exclude = {
 DataSourceAutoConfiguration.class,
 SecurityAutoConfiguration.class
})
public class MyApplication { }
```

### 3. @ComponentScan - Component Discovery

```
// Scans package and sub-packages for:
// @Component, @Service, @Repository, @Controller

// Default: scans package of @SpringBootApplication class
@SpringBootApplication
public class MyApplication { } // Scans com.example.* if class is in com.example

// Custom scan:
@SpringBootApplication(scanBasePackages = {
 "com.example.myapp",
 "com.example.common"
})
public class MyApplication { }

// Exclude specific components:
@SpringBootApplication(
 scanBasePackageClasses = {MyApplication.class},
 excludeFilters = @ComponentScan.Filter(
 type = FilterType.ASSIGNABLE_TYPE,
 classes = LegacyService.class
)
)
```

### Real Example - Multi-Module Project:

```
// Parent module: demo
// Sub-modules: common, evaluation-service, sampling-service

// evaluation-
service/src/main/java/com/example/evaluation/EvaluationServiceApplication.java
@SpringBootApplication(scanBasePackages = {
 "com.example.evaluation", // Scan this module
 "com.example.common" // Scan common module
})
@EnableJpaRepositories("com.example.evaluation.repository")
@EntityScan("com.example.evaluation.entity")
public class EvaluationServiceApplication { }
```



```

 public static void main(String[] args) {
 SpringApplication app = new
SpringApplication(EvaluationServiceApplication.class);

 // Set default properties
 app.setDefaultProperties(Map.of(
 "spring.application.name", "evaluation-service",
 "server.port", "8081"
));

 app.run(args);
 }
}
```
---

## Spring Boot Essentials

### Q13: "What is Spring Boot and how is it different from Spring?"

**Answer:**

**Spring Framework:**


- Core framework for DI, IoC, AOP
- Requires extensive XML or Java configuration
- Manual configuration for every component
- Complex setup for web applications



**Spring Boot:**


- Opinionated framework built on top of Spring
- Auto-configuration based on classpath
- Embedded servers (Tomcat, Jetty)
- Production-ready features (Actuator, Metrics)



**â€œ Traditional Spring Configuration:**
```xml
<!-- web.xml -->
<servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
</servlet>

<!-- applicationContext.xml -->
<beans>
 <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
 <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
 <property name="url" value="jdbc:mysql://localhost:3306/mydb"/>
 <property name="username" value="root"/>
 <property name="password" value="password"/>
 </bean>

 <bean id="sessionFactory"
class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">

```

```
 <property name="dataSource" ref="dataSource"/>
 </bean>

 <!-- 50+ more bean definitions... -->
</beans>
```

## **\*\* Spring Boot - Zero Configuration:\*\***

```
@SpringBootApplication // = @Configuration + @EnableAutoConfiguration +
@ComponentScan
public class MyApplication {
 public static void main(String[] args) {
 SpringApplication.run(MyApplication.class, args);
 }
}

// application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update

// That's it! Spring Boot auto-configures everything
```

## **Spring Boot Auto-Configuration Magic:**

```
// If spring-boot-starter-web on classpath:
// - Configures DispatcherServlet
// - Configures embedded Tomcat
// - Configures Jackson for JSON
// - Configures error handling

// If spring-boot-starter-data-jpa on classpath:
// - Configures DataSource
// - Configures EntityManagerFactory
// - Configures TransactionManager
// - Enables @Transactional

// If spring-boot-starter-security on classpath:
// - Configures security filters
// - Generates default password
// - Secures all endpoints
```

## **Spring Boot Starters:**

```
<!-- Single dependency includes everything needed -->
<dependency>
```

```

 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
 </dependency>

 <!-- Includes: spring-web, spring-webmvc, tomcat, jackson, validation -->

 <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-data-jpa</artifactId>
 </dependency>

 <!-- Includes: spring-data-jpa, hibernate, jdbc, transaction -->

 <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
 </dependency>

 <!-- Includes: spring-security-core, spring-security-web, spring-security-config -
 ->

```

### Real Example - My Microservice:

```

@SpringBootApplication
@EnableAsync
@EnableScheduling
@EnableCaching
public class EvaluationServiceApplication {

 public static void main(String[] args) {
 SpringApplication.run(EvaluationServiceApplication.class, args);
 }

 @Bean
 public RestTemplate restTemplate(RestTemplateBuilder builder) {
 return builder
 .setConnectTimeout(Duration.ofSeconds(5))
 .setReadTimeout(Duration.ofSeconds(10))
 .build();
 }
}

// application.yml - all configuration in one place
spring:
 application:
 name: evaluation-service
 datasource:
 url: jdbc:postgresql://localhost:5432/evaluationdb
 username: ${DB_USERNAME:admin}
 password: ${DB_PASSWORD:password}
 jpa:
 hibernate:

```

```

 ddl-auto: validate
 show-sql: false
kafka:
 bootstrap-servers: localhost:9092
 consumer:
 group-id: evaluation-group
 producer:
 key-serializer: org.apache.kafka.common.serialization.StringSerializer
 value-serializer: org.apache.kafka.common.serialization.StringSerializer

server:
 port: 8081

management:
 endpoints:
 web:
 exposure:
 include: health,info,metrics,prometheus
 ""

Spring Boot Configuration & Profiles

Q18: "Explain application.properties vs application.yml and Profiles"

Answer:
```application.properties
```properties
Server configuration
server.port=8080
server.servlet.context-path=/api

Database
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=admin
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true

Kafka
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=my-group
spring.kafka.consumer.auto-offset-reset=earliest

Logging
logging.level.root=INFO
logging.level.com.example.myapp=DEBUG
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} - %msg%n

```

### application.yml (Preferred - More Readable)

```
server:
 port: 8080
 servlet:
 context-path: /api

spring:
 application:
 name: evaluation-service

 datasource:
 url: jdbc:postgresql://localhost:5432/evaluationdb
 username: ${DB_USERNAME:admin}
 password: ${DB_PASSWORD:secret}
 hikari:
 maximum-pool-size: 20
 minimum-idle: 5
 connection-timeout: 30000

 jpa:
 hibernate:
 ddl-auto: validate
 show-sql: false
 properties:
 hibernate:
 format_sql: true
 dialect: org.hibernate.dialect.PostgreSQLDialect

 kafka:
 bootstrap-servers: ${KAFKA_SERVERS:localhost:9092}
 consumer:
 group-id: evaluation-group
 auto-offset-reset: earliest
 producer:
 key-serializer: org.apache.kafka.common.serialization.StringSerializer
 value-serializer: org.apache.kafka.common.serialization.StringSerializer

 logging:
 level:
 root: INFO
 com.example.evaluation: DEBUG
 org.hibernate.SQL: DEBUG
 pattern:
 console: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n"
```

### Profile-Specific Configuration:

```
application.yml (default)
spring:
 application:
 name: evaluation-service
 profiles:
```

```
 active: ${SPRING_PROFILE:dev}

server:
 port: 8080

application-dev.yml
spring:
 config:
 activate:
 on-profile: dev

 datasource:
 url: jdbc:h2:mem:testdb
 username: sa
 password:

 jpa:
 hibernate:
 ddl-auto: create-drop
 show-sql: true

logging:
 level:
 root: DEBUG

application-staging.yml
spring:
 config:
 activate:
 on-profile: staging

 datasource:
 url: jdbc:postgresql://staging-db:5432/evaluationdb
 username: ${DB_USERNAME}
 password: ${DB_PASSWORD}

 jpa:
 hibernate:
 ddl-auto: validate
 show-sql: false

logging:
 level:
 root: INFO
 com.example: DEBUG

application-prod.yml
spring:
 config:
 activate:
 on-profile: prod
```

```
datasource:
 url: jdbc:postgresql://prod-db:5432/evaluationdb
 username: ${DB_USERNAME}
 password: ${DB_PASSWORD}
 hikari:
 maximum-pool-size: 50
 minimum-idle: 10

jpa:
 hibernate:
 ddl-auto: validate
 show-sql: false

logging:
 level:
 root: WARN
 com.example: INFO

management:
 endpoints:
 web:
 exposure:
 include: health,metrics,prometheus
```

### Using Profiles:

```
Command line
java -jar app.jar --spring.profiles.active=prod

Environment variable
export SPRING_PROFILES_ACTIVE=prod
java -jar app.jar

In application.yml
spring:
 profiles:
 active: dev

Multiple profiles
java -jar app.jar --spring.profiles.active=prod,metrics
```

### @Profile Annotation:

```
@Configuration
@Profile("dev")
public class DevConfig {

 @Bean
```

```

 public DataSource dataSource() {
 return new EmbeddedDatabaseBuilder()
 .setType(EmbeddedDatabaseType.H2)
 .build();
 }
}

@Configuration
@Profile("prod")
public class ProdConfig {

 @Bean
 public DataSource dataSource() {
 HikariConfig config = new HikariConfig();
 config.setJdbcUrl(dbUrl);
 config.setMaximumPoolSize(50);
 return new HikariDataSource(config);
 }
}

@Service
@Profile("!prod") // Active when NOT prod
public class MockEmailService implements EmailService {
 @Override
 public void send(String to, String message) {
 logger.info("Mock email to {}: {}", to, message);
 }
}

@Service
@Profile("prod")
public class RealEmailService implements EmailService {
 @Override
 public void send(String to, String message) {
 // Actually send email
 sesClient.sendEmail(to, message);
 }
}

```

### @ConfigurationProperties:

```

@Data
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
 private String name;
 private String version;
 private Security security = new Security();
 private Kafka kafka = new Kafka();

 @Data
 public static class Security {

```



```

 private String jwtSecret;
 private long jwtExpiration;
 }

 @Data
 public static class Kafka {
 private String bootstrapServers;
 private String topic;
 }
}

application.yml
app:
 name: Evaluation Service
 version: 1.0.0
 security:
 jwt-secret: ${JWT_SECRET:mySecretKey}
 jwt-expiration: 86400000
 kafka:
 bootstrap-servers: localhost:9092
 topic: evaluations

@Service
public class MyService {
 private final AppProperties properties;

 public MyService(AppProperties properties) {
 this.properties = properties;
 }

 public void doSomething() {
 String jwtSecret = properties.getSecurity().getJwtSecret();
 String kafkaTopic = properties.getKafka().getTopic();
 }
}
```

---

## ÖY”€ JWT & Spring Security Questions

### Q1: "How do you implement JWT authentication in Spring Boot?"

**Answer:**

"I secure REST APIs in Spring Boot using Spring Security with JWT-based authentication:

**Step 1: Dependencies**

```xml
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>

```

```
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.12.3</version>
</dependency>
```

**Step 2: JWT Token Provider** I created `JwtTokenProvider` class that:

- Generates tokens with username and roles
- Validates tokens using HMAC-SHA256
- Extracts claims (username, roles, expiry)

**Step 3: Authentication Filter** `JwtAuthenticationFilter` extends `OncePerRequestFilter`:

- Intercepts every request
- Extracts JWT from Authorization header (`Bearer <token>`)
- Validates token and sets authentication in `SecurityContext`

**Step 4: Security Configuration** `SecurityConfig` configures:

- Stateless session management (no `HttpSession`)
- Public endpoints (login, health checks, Swagger)
- Protected endpoints with role-based authorization
- Custom authentication entry point for 401 errors

**Step 5: Authentication Flow**

```
User POST /api/auth/login Validate credentials
 Generate JWT with roles Return token
User GET /api/users (Header: Bearer <token>)
 Filter extracts token Validate Set authentication
 Controller checks @PreAuthorize Process request
```

**In my project, I implemented this in the `common` module so all microservices inherit JWT security."**

---

Q2: "What's the difference between stateless and stateful authentication?"

**Answer: "Stateful (Traditional Session-based):"**

```
// Problem: Server stores session state
public class UserService {
 public void login(String username, String password) {
 // Authenticate user
 // Create HttpSession
 // Store user details in session
 }

 public User getCurrentUser() {
 // Retrieve user from HttpSession
 }
}
```

```
}
}
```

**Issues:**

- Session data in-memory or database
- Memory consumption grows with active users
- Harder to scale (session replication)

---

**Stateless (JWT-based):**

```
public class UserService {
 public String login(String username, String password) {
 // Authenticate user
 // Generate JWT token with user details
 return token;
 }

 public User getCurrentUser(String token) {
 // Validate token
 // Extract user details from token
 }
}
```

**Benefits:**

- No session data stored on server
- Scales effortlessly (just validate token)
- Reduced memory footprint

**Trade-offs:**

- Token size impacts performance
- Revoking tokens is complex (short-lived tokens help)
- Must handle token expiration and refresh

**My Implementation:**

- Stateless JWT authentication
- Refresh tokens stored in Redis (for revocation and refresh)
- Short-lived access tokens (15 min), long-lived refresh tokens (7 days)"

---

Q3: "How do you handle JWT token expiration?"

**Answer:** "I handle token expiration with a multi-layered approach:

**1. Token Expiry Configuration:**

```
jwt.expiration=86400000 # 24 hours
```

## 2. Validation in JwtTokenProvider:

```
public boolean validateToken(String token) {
 try {
 Jwts.parserBuilder()
 .setSigningKey(key)
 .build()
 .parseClaimsJws(token); // Throws ExpiredJwtException if expired
 return true;
 } catch (ExpiredJwtException ex) {
 logger.error("Expired JWT token");
 return false;
 }
}
```

## 3. Client-side Handling:

- Frontend stores expiry time from login response
- Shows 'Session expired' dialog before making requests
- Automatically redirects to login when 401 received

## 4. Refresh Token Pattern (Production):

- Issue short-lived access token (15 min)
- Issue long-lived refresh token (7 days)
- POST /api/auth/refresh with refresh token
- Get new access token without re-login

## 5. Security Consideration:

- Tokens in **Authorization: Bearer** header (not localStorage for XSS protection)
- Use HttpOnly cookies for refresh tokens
- Implement token blacklist for logout (Redis)

**In my project, I set 24-hour expiry for development, but would use 15-minute with refresh tokens in production."**

---

Q4: "Explain role-based authorization vs permission-based authorization"

**Answer: "Role-Based Access Control (RBAC):"**

```
@Service
public class UserService {
 // Only admins can access
 @PreAuthorize("hasRole('ADMIN')")
```

```
public void deleteUser(Long id) {
 userRepository.deleteById(id);
}
}
```

**Pros:**

- Simple, easy to understand
- Roles map to real-world job functions

**Cons:**

- Roles can become too granular (role explosion)
  - Hard to manage permissions at a fine level
- 

**Permission-Based (Fine-grained):**

```
@Service
public class DocumentService {
 // Specific permission required
 @PreAuthorize("hasAuthority('DOCUMENT_EDIT')")
 public void editDocument(Long docId, DocumentUpdateRequest request) {
 // Edit document logic
 }
}
```

**Pros:**

- Very flexible, precise control
- Can model complex access rules

**Cons:**

- More complex to manage
- Requires careful design to avoid explosion of permissions

**My Implementation:**

- Used RBAC for simplicity
- Admins have full access, users have limited access
- In future, may migrate to permission-based for finer control

**Hybrid Approach:**

- Use roles for common access patterns
- Use permissions for exceptions or special cases
- Example: Admin role has all permissions, but can be restricted per user

```

```

```
âš™i, □ AWS & Terraform Questions
```

```
Q5: "How do you deploy Spring Boot microservices to AWS?"
```

```
Answer:
```

```
"I use **Infrastructure as Code (Terraform)** for fully automated deployment:
```

```
Architecture:
```

Terraform Creates: â€ VPC with public/private subnets (multi-AZ) â€ RDS PostgreSQL (managed database)  
 â€ ECR repositories (Docker images) â€ ECS Fargate cluster (serverless containers) â€ Application Load  
 Balancer â€ Lambda functions â€ Step Functions (workflow orchestration) â€ CloudWatch Logs & Alarms  
 â€ IAM roles & Security Groups

```
Deployment Process:
```

```
```bash
```

```
# 1. Build Spring Boot apps
```

```
mvn clean package -DskipTests
```

```
# 2. Build Docker images
```

```
docker build -t evaluation-service:latest ./evaluation-service
```

```
# 3. Push to ECR
```

```
aws ecr get-login-password | docker login ...
```

```
docker tag evaluation-service:latest <ecr-url>/evaluation-service:latest
```

```
docker push <ecr-url>/evaluation-service:latest
```

```
# 4. Deploy infrastructure (one command!)
```

```
cd terraform
```

```
terraform init
```

```
terraform apply -auto-approve
```

```
# 5. ECS automatically pulls from ECR and runs containers
```

Key Terraform Files:

- `main.tf` - VPC, networking
- `ecs.tf` - Container orchestration
- `rds.tf` - PostgreSQL database
- `lambda.tf` - Serverless functions
- `ecr.tf` - Docker registries

Benefits:

- Zero manual AWS console clicks

- Reproducible (dev, staging, prod identical)
- Version controlled (Git)
- Automated scaling
- Cost-effective (Fargate auto-scaling)

In my project, **terraform apply** creates 50+ AWS resources in 15 minutes."

Q6: "How do you manage database migrations in microservices?"

Answer: "I use **Flyway** or **Liquibase** for version-controlled database migrations:

Flyway Implementation:

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
```

Migration Files:

```
src/main/resources/db/migration/
â”€ V1__create_users_table.sql
â”€ V2__add_email_index.sql
â”€â”€â”€ V3__add_roles_table.sql
```

V1__create_users_table.sql:

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(100) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
```

Spring Boot Configuration:

```
spring:
  flyway:
    enabled: true
    baseline-on-migrate: true
    locations: classpath:db/migration
    schemas: evaluation
```

Migration Strategy:

1. **Development:** `flyway migrate` on startup
2. **Production:** Manual review `flyway migrate` in CI/CD
3. **Rollback:** Flyway supports undo migrations

Per-Service Schema Isolation:

```
-- Each microservice has its own schema
CREATE SCHEMA evaluation;
CREATE SCHEMA sampling;
CREATE SCHEMA evidence;
CREATE SCHEMA remediation;
CREATE SCHEMA jira_service;
```

In my project:

- RDS PostgreSQL with separate schemas per service
- Flyway runs automatically on ECS container startup
- Migration history in `flyway_schema_history` table
- Terraform creates database, Flyway manages schema

Benefits:

- Version controlled SQL
- Repeatable deployments
- Rollback capability
- Team collaboration (no manual DDL scripts)"

½ Microservices Communication Patterns

Q7A: "How do microservices communicate with each other in your architecture?"

Answer: "In my microservices architecture, I implement **4 main communication patterns** based on the use case:

1. Synchronous REST API (Most Common)

```
@Service
public class TransactionService {
    @Autowired
    private RestTemplate restTemplate;

    public TransactionResponse processTransaction(TransactionRequest request) {
        // Call Account Service to check balance
        String url = "http://account-service/api/accounts/" +
request.getAccountId();
        AccountResponse account = restTemplate.getForObject(url,
```



```
AccountResponse.class);

    if (account.getBalance() < request.getAmount()) {
        throw new InsufficientFundsException();
    }

    // Process transaction
    return performTransaction(request);
}
}
```

When to use: Need immediate response (account balance check, payment validation)

2. Asynchronous Messaging (SNS/SQS, Kafka)

```
@Service
public class OrderService {
    @Autowired
    private SnsClient snsClient;

    public OrderResponse createOrder(OrderRequest request) {
        // Save order to database
        Order order = orderRepository.save(request);

        // Send async notification (fire and forget)
        PublishRequest publishRequest = PublishRequest.builder()
            .topicArn("arn:aws:sns:us-east-1:123456789:order-notifications")
            .message(objectMapper.writeValueAsString(order))
            .build();
        snsClient.publish(publishRequest);

        return new OrderResponse(order);
    }
}

@Service
public class NotificationService {
    @SqsListener("order-notifications-queue")
    public void handleOrderCreated(OrderMessage message) {
        // Async processing - send email, SMS
        emailService.send(message.getCustomerEmail(), "Order Confirmed");
    }
}
```

When to use: Non-critical operations (notifications, logging, analytics)

3. Service Discovery Pattern

```
@Configuration
@EnableDiscoveryClient
```

When to use: Production with auto-scaling (IP addresses change dynamically)

66 / 170


```
gateway:
  routes:
    - id: user-service
      uri: lb://user-service
      predicates:
        - Path=/api/users/**
      filters:
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenishRate: 10 # 10 requests per second
            redis-rate-limiter.burstCapacity: 20 # Max 20 requests burst
```

4. Request/Response Transformation:

```
@Component
public class AddHeaderFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        // Add custom headers
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header("X-Gateway-Request-Id", UUID.randomUUID().toString())
            .header("X-Gateway-Timestamp", Instant.now().toString())
            .build();

        return chain.filter(exchange.mutate().request(request).build());
    }
}
```

5. Load Balancing:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
```

6. Circuit Breaker Integration:

```
spring:
  cloud:
    gateway:
      routes:
```

```

- id: payment-service
  uri: lb://payment-service
  predicates:
    - Path=/api/payments/**
  filters:
    - name: CircuitBreaker
      args:
        name: paymentCircuitBreaker
        fallbackUri: forward:/fallback/payment

```

7. SSL Termination:

```

HTTPS (SSL/TLS)
    â†“
API Gateway (handles SSL)
    â†“
HTTP (internal network, no SSL overhead)
    â†“
Microservices

```

Spring Cloud Gateway Implementation:

```

@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            // User Service
            .route("user-service", r -> r
                .path("/api/users/**")
                .filters(f -> f
                    .addRequestHeader("X-Source", "API-Gateway")
                    .circuitBreaker(c -> c
                        .setName("userServiceCircuitBreaker")
                        .setFallbackUri("forward:/fallback/users")
                    )
                )
            .uri("lb://user-service")
        )

        // Order Service with rate limiting
        .route("order-service", r -> r
            .path("/api/orders/**")
            .filters(f -> f
                .requestRateLimiter(c -> c
                    .setRateLimiter(redisRateLimiter())
                )
            )
        )
    }
}

```

```
        )
        .uri("lb://order-service")
    )

    // Payment Service (secure, no rate limit)
    .route("payment-service", r -> r
        .path("/api/payments/**")
        .filters(f -> f
            .filter(jwtAuthenticationFilter)
        )
        .uri("lb://payment-service")
    )

    .build();
}
}
```

API Gateway vs Service Mesh:

Feature	API Gateway	Service Mesh
Layer	Application Layer (L7)	Network Layer (L4-L7)
Scope	External traffic (clients services)	Internal traffic (service service)
Example	Spring Cloud Gateway, AWS API Gateway	Istio, Linkerd, AWS App Mesh
Use Case	Public API, authentication, rate limiting	Internal communication, observability

In Production: Use both!

- API Gateway for external traffic
- Service Mesh for internal service-to-service

AWS API Gateway (Serverless Option):

```
# terraform/api-gateway.tf
resource "aws_api_gateway_rest_api" "main" {
    name = "microservices-api"
}

resource "aws_api_gateway_resource" "users" {
    rest_api_id = aws_api_gateway_rest_api.main.id
    parent_id   = aws_api_gateway_rest_api.main.root_resource_id
    path_part   = "users"
}

resource "aws_api_gateway_method" "get_users" {
    rest_api_id = aws_api_gateway_rest_api.main.id
    resource_id = aws_api_gateway_resource.users.id
```

```

    http_method    = "GET"
    authorization = "CUSTOM"
    authorizer_id = aws_api_gateway_authorizer.jwt.id
  }

  resource "aws_api_gateway_integration" "user_service" {
    rest_api_id = aws_api_gateway_rest_api.main.id
    resource_id = aws_api_gateway_resource.users.id
    http_method = aws_api_gateway_method.get_users.http_method

    type                = "HTTP_PROXY"
    integration_http_method = "GET"
    uri                 = "http://user-service.internal/api/users"
  }

```

Benefits of AWS API Gateway:

- Fully managed (no servers to maintain)
- Auto-scaling (handles millions of requests)
- Built-in throttling and quotas
- Request/response transformation
- AWS WAF integration (DDoS protection)

ðŸ›¸ Circuit Breaker Patterns

Q14: "Explain Circuit Breaker pattern and why it's important"

Answer: "**Circuit Breaker** prevents cascading failures in microservices by stopping calls to failing services.

Real-World Analogy:

- Your home's circuit breaker
- If electrical fault detected Cut power immediately
- Prevents fire/damage
- After some time Try again (reset)

The Problem: Cascading Failure

```

User Service calls Payment Service
    â†’
Payment Service is DOWN (crashed)
    â†’
User Service keeps trying (10 requests/sec)
    â†’
Each request waits 30 seconds (timeout)
    â†’
User Service threads exhausted (all waiting)
    â†’
User Service crashes too!

```


2. Authentication:

```
@Component
public class JwtAuthenticationFilter implements GatewayFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        String token =
        exchange.getRequest().getHeaders().getFirst("Authorization");

        if (token == null || !jwtTokenProvider.validateToken(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // Add user info to request headers
        String username = jwtTokenProvider.getUsername(token);
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header("X-User-Name", username)
            .build();

        return chain.filter(exchange.mutate().request(request).build());
    }
}
```

Now backend services don't need JWT validation!

3. Rate Limiting:

```
# application.yml
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10 # 10 requests per second
                redis-rate-limiter.burstCapacity: 20 # Max 20 requests burst
```

4. Request/Response Transformation:

```
@Component
public class AddHeaderFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        // Add custom headers
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header("X-Gateway-Request-Id", UUID.randomUUID().toString())
            .header("X-Gateway-Timestamp", Instant.now().toString())
            .build();

        return chain.filter(exchange.mutate().request(request).build());
    }
}
```

5. Load Balancing:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
```

6. Circuit Breaker Integration:

```
spring:
  cloud:
    gateway:
      routes:
        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**
          filters:
            - name: CircuitBreaker
              args:
                name: paymentCircuitBreaker
                fallbackUri: forward:/fallback/payment
```

7. SSL Termination:

```
HTTPS (SSL/TLS)
    â†“
    API Gateway (handles SSL)
    â†“
    HTTP (internal network, no SSL overhead)
    â†“
    Microservices
```

Spring Cloud Gateway Implementation:

```
@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            // User Service
            .route("user-service", r -> r
                .path("/api/users/**")
                .filters(f -> f
                    .addRequestHeader("X-Source", "API-Gateway")
                    .circuitBreaker(c -> c
                        .setName("userServiceCircuitBreaker")
                        .setFallbackUri("forward:/fallback/users")
                    )
                )
                .uri("lb://user-service")
            )

            // Order Service with rate limiting
            .route("order-service", r -> r
                .path("/api/orders/**")
                .filters(f -> f
                    .requestRateLimiter(c -> c
                        .setRateLimiter(redisRateLimiter())
                    )
                )
                .uri("lb://order-service")
            )

            // Payment Service (secure, no rate limit)
            .route("payment-service", r -> r
                .path("/api/payments/**")
                .filters(f -> f
                    .filter(jwtAuthenticationFilter)
                )
                .uri("lb://payment-service")
            )

        .build();
    }
}
```

```
    }  
  }  
}
```

API Gateway vs Service Mesh:

Feature	API Gateway	Service Mesh
Layer	Application Layer (L7)	Network Layer (L4-L7)
Scope	External traffic (clients services)	Internal traffic (service service)
Example	Spring Cloud Gateway, AWS API Gateway	Istio, Linkerd, AWS App Mesh
Use Case	Public API, authentication, rate limiting	Internal communication, observability

In Production: Use both!

- API Gateway for external traffic
- Service Mesh for internal service-to-service

AWS API Gateway (Serverless Option):

```
# terraform/api-gateway.tf  
resource "aws_api_gateway_rest_api" "main" {  
  name = "microservices-api"  
}  
  
resource "aws_api_gateway_resource" "users" {  
  rest_api_id = aws_api_gateway_rest_api.main.id  
  parent_id   = aws_api_gateway_rest_api.main.root_resource_id  
  path_part   = "users"  
}  
  
resource "aws_api_gateway_method" "get_users" {  
  rest_api_id      = aws_api_gateway_rest_api.main.id  
  resource_id      = aws_api_gateway_resource.users.id  
  http_method      = "GET"  
  authorization    = "CUSTOM"  
  authorizer_id    = aws_api_gateway_authorizer.jwt.id  
}  
  
resource "aws_api_gateway_integration" "user_service" {  
  rest_api_id = aws_api_gateway_rest_api.main.id  
  resource_id = aws_api_gateway_resource.users.id  
  http_method = aws_api_gateway_method.get_users.http_method  
  
  type                = "HTTP_PROXY"  
  integration_http_method = "GET"  
  uri                 = "http://user-service.internal/api/users"  
}
```

Benefits of AWS API Gateway:

- Fully managed (no servers to maintain)
- Auto-scaling (handles millions of requests)
- Built-in throttling and quotas
- Request/response transformation
- AWS WAF integration (DDoS protection)

ðŸ›¸ ðŸ“œ Circuit Breaker Patterns

Q14: "Explain Circuit Breaker pattern and why it's important"

Answer: "**Circuit Breaker** prevents cascading failures in microservices by stopping calls to failing services.

Real-World Analogy:

- Your home's circuit breaker
- If electrical fault detected Cut power immediately
- Prevents fire/damage
- After some time Try again (reset)

The Problem: Cascading Failure

```
User Service calls Payment Service
    â†“
Payment Service is DOWN (crashed)
    â†“
User Service keeps trying (10 requests/sec)
    â†“
Each request waits 30 seconds (timeout)
    â†“
User Service threads exhausted (all waiting)
    â†“
User Service crashes too!
    â†“
Order Service (depends on User Service) crashes
    â†“
ENTIRE SYSTEM DOWN ðŸ›¸
```

Impact:

- 10 requests/sec Ã— 30 sec timeout = 300 threads blocked
- Thread pool exhausted in 30 seconds
- Domino effect: 1 service failure All services fail

Solution: Circuit Breaker


```

    @Autowired
    private ServiceA serviceA;

    @Autowired
    private ServiceB serviceB;

    @Autowired
    private ServiceC serviceC;

    public void processPayment() {
        // Use service name (not IP address)
        String url = "http://account-service/api/accounts/123";
        // Service Discovery resolves to: http://10.0.1.45:8081/api/accounts/123
        Account account = restTemplate.getForObject(url, Account.class);
    }
}

```

When to use: Non-critical operations (notifications, logging, analytics)

3. Service Discovery Pattern

```

@Configuration
@EnableDiscoveryClient
public class ServiceDiscoveryConfig {
    @Bean
    @LoadBalanced // Enables service name resolution
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@Service
public class PaymentService {
    @Autowired
    private RestTemplate restTemplate;

    public void processPayment() {
        // Use service name (not IP address)
        String url = "http://account-service/api/accounts/123";
        // Service Discovery resolves to: http://10.0.1.45:8081/api/accounts/123
        Account account = restTemplate.getForObject(url, Account.class);
    }
}

```

When to use: Production with auto-scaling (IP addresses change dynamically)

4. API Gateway Pattern

```

Mobile App / Browser
  â†“
  API Gateway (8080)
    â†“
    â”€ /api/accounts/* Account Service (8081)
    â”€ /api/transactions/* Transaction Service (8082)
    â”€ /api/loans/* Loan Service (8083)
    â”€â”€â”€ /api/notifications/* Notification Service (8084)

```

Benefits:

- Single entry point for clients
- Centralized authentication (JWT validation)
- Rate limiting and throttling
- Request/response transformation
- SSL termination

In my project:

- Account Service Transaction Service: Synchronous REST (need balance immediately)
- Order Service Notification Service: Async SNS/SQS (can wait for email)
- All external clients API Gateway Services
- Services discover each other via AWS Cloud Map (Service Discovery)"

Q7B: "What's the difference between tight coupling and loose coupling?"

Answer: "Tight Coupling (BAD):

```

// Transaction Service directly instantiates Account Service
public class TransactionService {
    public void processTransaction() {
        AccountService accountService = new AccountService();
        accountService.debitAccount(accountId, amount);
        // Problem: Can't deploy Transaction Service without Account Service!
    }
}

```

Problems:

- â□€ Can't deploy services independently
- â□€ If Account Service changes, Transaction Service breaks
- â□€ Can't scale services separately
- â□€ Hard to test in isolation

Loose Coupling (GOOD):

```
// Transaction Service calls Account Service via API
public class TransactionService {
    @Autowired
    private RestTemplate restTemplate;

    public void processTransaction() {
        String url = "http://account-service/api/accounts/" + accountId +
"/debit";
        restTemplate.postForObject(url, request, Response.class);
        // Services are independent!
    }
}
```

Benefits:

- Services deployed independently
- Technology agnostic (Java, Python, Node.js)
- Scale independently
- Easy to mock for testing

Key Principle:

- Services communicate via **contracts (APIs)**
- Services don't share code/libraries (except common DTOs)
- Each service has its own database (database per service pattern)"

☹️ RestTemplate vs WebClient vs Feign Client

Q8: "What's the difference between RestTemplate, WebClient, and Feign Client? When do you use each?"

Answer: "I've used all three in production, and here's my comparison:

1. RestTemplate (Legacy, Blocking)

Characteristics:

- Synchronous (blocking) - 1 thread per request
- Simple and straightforward
- ☹️ In maintenance mode (Spring recommends WebClient)

Example:

```
@Service
public class PaymentService {
    @Autowired
    private RestTemplate restTemplate;

    public PaymentResponse processPayment(Long orderId) {
```

```

        // Blocks thread until response received
        String url = "http://payment-gateway/api/payments";
        PaymentRequest request = new PaymentRequest(orderId);
        PaymentResponse response = restTemplate.postForObject(url, request,
            PaymentResponse.class);
        return response;
    }
}

```

Performance:

- 1000 concurrent requests = 1000 threads needed
- Each thread = ~1MB memory
- Total: ~1GB just for threads

When to use:

- Legacy Spring Boot projects (already using it)
- Simple synchronous calls
- ❌ NOT for new projects

2. WebClient (Modern, Non-Blocking)

Characteristics:

- **Asynchronous (reactive)** - Few threads handle many requests
- Non-blocking I/O
- **Spring's official recommendation**
- Supports both sync and async modes

Example - Synchronous Mode:

```

@Service
public class PaymentService {
    @Autowired
    private WebClient webClient;

    public PaymentResponse processPayment(Long orderId) {
        return webClient.post()
            .uri("/api/payments")
            .bodyValue(new PaymentRequest(orderId))
            .retrieve()
            .bodyToMono(PaymentResponse.class)
            .block(); // Block for synchronous behavior
    }
}

```

Example - Asynchronous Mode (Better):

```

@Service
public class OrderService {
    @Autowired
    private WebClient webClient;

    public Mono<OrderResponse> createOrder(OrderRequest request) {
        // Call 3 services in parallel (non-blocking)
        Mono<Product> productMono = webClient.get()
            .uri("/api/products/" + request.getProductId())
            .retrieve()
            .bodyToMono(Product.class);

        Mono<Inventory> inventoryMono = webClient.get()
            .uri("/api/inventory/" + request.getProductId())
            .retrieve()
            .bodyToMono(Inventory.class);

        Mono<User> userMono = webClient.get()
            .uri("/api/users/" + request.getUserId())
            .retrieve()
            .bodyToMono(User.class);

        // Combine all results
        return Mono.zip(productMono, inventoryMono, userMono)
            .map(tuple -> {
                Product product = tuple.getT1();
                Inventory inventory = tuple.getT2();
                User user = tuple.getT3();
                return processOrder(product, inventory, user);
            });
    }
}

```

Performance:

- 1000 concurrent requests = ~10-20 threads (reused)
- Each thread handles 50-100 requests
- Total: ~20MB memory (**50x more efficient!**)

Configuration:

```

@Configuration
public class WebClientConfig {
    @Bean
    public WebClient webClient() {
        return WebClient.builder()
            .baseUrl("http://payment-gateway")
            .defaultHeader("Content-Type", "application/json")
            .defaultHeader("User-Agent", "OrderService")
            .filter(ExchangeFilterFunction.ofRequestProcessor(clientRequest -> {
                logger.info("Calling: {}", clientRequest.url());
            }));
    }
}

```

```

        return Mono.just(clientRequest);
    }
}

```

When to use:

- **NEW Spring Boot projects (highly recommended)**
- High-traffic applications (10,000+ req/sec)
- Need to call multiple APIs in parallel
- Building reactive applications

3. Feign Client (Declarative, Easy)

Characteristics:

- Declarative (write interface, no implementation)
- Built-in load balancing with Spring Cloud
- Easy integration with Circuit Breaker
- Great for microservices communication

Example:

```

// Just define interface - NO implementation needed!
@FeignClient(
    name = "payment-gateway",
    url = "http://payment-gateway:8080",
    fallback = PaymentGatewayFallback.class
)
public interface PaymentGatewayClient {

    @PostMapping("/api/payments")
    PaymentResponse processPayment(@RequestBody PaymentRequest request);

    @GetMapping("/api/payments/{id}")
    PaymentResponse getPayment(@PathVariable("id") Long id);
}

// Fallback when service is down
@Component
public class PaymentGatewayFallback implements PaymentGatewayClient {
    @Override
    public PaymentResponse processPayment(PaymentRequest request) {
        return new PaymentResponse("PENDING", "Service unavailable");
    }

    @Override
    public PaymentResponse getPayment(Long id) {
        return new PaymentResponse("UNKNOWN", "Service unavailable");
    }
}

```

```

    }
}

// Use in service (super simple!)
@Service
public class OrderService {
    @Autowired
    private PaymentGatewayClient paymentClient;

    public OrderResponse createOrder(OrderRequest request) {
        // Just call the method - Feign handles everything!
        PaymentResponse payment = paymentClient.processPayment(
            new PaymentRequest(request.getAmount())
        );
        return new OrderResponse(payment);
    }
}

```

Configuration:

```

@SpringBootApplication
@EnableFeignClients // Enable Feign
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}

```

Advanced Features:

```

# application.yml
feign:
  client:
    config:
      default:
        connectTimeout: 5000
        readTimeout: 10000
        loggerLevel: full # Log all requests/responses
  circuitbreaker:
    enabled: true

resilience4j:
  circuitbreaker:
    instances:
      payment-gateway:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 60s

```

When to use:

- **Microservices calling each other (best choice)**
- Want declarative style (less code)
- Need built-in Circuit Breaker
- Want automatic retry logic
- Spring Cloud ecosystem

Comparison Table:

Feature	RestTemplate	WebClient	Feign Client
Type	Blocking	Non-blocking	Blocking
Performance	Good	Excellent (10x)	Good
Code Style	Imperative	Reactive	Declarative
Learning Curve	Easy	Medium	Easy
Spring Recommendation	☐ Maintenance	Preferred	Good
Boilerplate Code	Medium	High	Low
Circuit Breaker	Manual	Manual	Built-in
Load Balancing	Manual	Manual	Built-in
Retry Logic	Manual	Manual	Built-in
Best For	Legacy	High-performance	Microservices

My Production Experience:**Scenario 1: E-Commerce Checkout**

- Used **WebClient** to call Product, Inventory, Payment APIs in parallel
- 3 API calls in 2 seconds (instead of 6 seconds sequential)
- Result: 3x faster checkout

Scenario 2: Internal Microservices

- Used **Feign Client** for Order Service Inventory Service
- Built-in retry when Inventory Service was down
- Fallback returned cached inventory data
- Result: Zero downtime during deployments

Scenario 3: Legacy System Integration

- Used **RestTemplate** to call old SOAP service
- Simple, worked fine for low traffic
- Result: Good enough for 100 req/min

My Recommendation:

For NEW projects:

```
â”€ Internal microservices communication Feign Client (easiest)
â”€ High-traffic APIs (>1000 req/sec) WebClient (fastest)
â”€ Parallel API calls WebClient (non-blocking)
â””â”€â”€ Simple REST calls WebClient (future-proof)
```

For EXISTING projects:

```
â”€ Already using RestTemplate? Keep it (don't refactor unless needed)
â””â”€â”€ Performance issues? Migrate to WebClient
```
```

---

## âš¡ Async Programming & @Async Annotation

### Q9: "Explain synchronous vs asynchronous programming with real examples"

**\*\*Answer:\*\***

**\*\*\*Synchronous (Blocking):\*\***

**\*\*Real-World Analogy:\*\***

- You order food at McDonald's
- You WAIT at counter until food is ready
- You can't do anything else (blocked)

**\*\*Code Example:\*\***

```
```java
@RestController
public class OrderController {
    @Autowired
    private EmailService emailService;

    @PostMapping("/api/orders")
    public ResponseEntity<OrderResponse> createOrder(@RequestBody OrderRequest
request) {
        Order order = orderService.save(request);

        emailService.sendOrderConfirmation(order); // BLOCKS for 2 seconds
        smsService.sendOrderSms(order);           // BLOCKS for 1 second

        return ResponseEntity.ok(new OrderResponse(order));
        // Total time: 2s + 1s = 3 seconds (user waits!)
    }
}
```

Problems:

- â”€ User waits 3 seconds for response
- â”€ Thread is blocked (wasted)

- Under load, threads exhausted
 - Poor user experience
-

Asynchronous (Non-Blocking):

Real-World Analogy:

- You order food at McDonald's
- They give you a buzzer
- You sit down, browse phone (free to do other things)
- Buzzer vibrates when food is ready

Code Example:

```
@RestController
public class OrderController {
    @Autowired
    private NotificationService notificationService;

    @PostMapping("/api/orders")
    public ResponseEntity<OrderResponse> createOrder(@RequestBody OrderRequest
request) {
        Order order = orderService.save(request);

        // Fire and forget (non-blocking)
        notificationService.sendOrderNotifications(order);

        return ResponseEntity.ok(new OrderResponse(order));
        // Total time: <100ms (instant response!)
    }
}

@Service
public class NotificationService {

    @Async // Runs in separate thread
    public void sendOrderNotifications(Order order) {
        emailService.sendOrderConfirmation(order); // 2 seconds
        smsService.sendOrderSms(order);           // 1 second
        // User doesn't wait for this!
    }
}
```

Benefits:

- User gets instant response (<100ms)
 - Thread is freed immediately
 - Notifications sent in background
 - Better scalability
-

Spring @Async Configuration:

```

@Configuration
@EnableAsync // Enable async support
public class AsyncConfig implements AsyncConfigurer {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);           // 10 threads always available
        executor.setMaxPoolSize(50);            // Max 50 threads under load
        executor.setQueueCapacity(100);         // Queue 100 tasks
        executor.setThreadNamePrefix("Async-");
        executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
        executor.initialize();
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (throwable, method, params) -> {
            logger.error("Async error in {}: {}", method.getName(),
throwable.getMessage());
        };
    }
}

```

Visual Comparison:**Synchronous:**

```

Thread-1: [Create Order]â”€[Send Emailâ”€â”€â”€2sâ”€â”€â”€]â”€[Send
SMSâ”€â”€1sâ”€]â”€[Response] = 3s
Thread-2:                                     [Create Order]â”€[Send Email...
Thread-3:                                     [Create Order...

10 requests = 10 threads (exhausted!)

```

Asynchronous:

```

Thread-1: [Create Order]â”€[Response]â”€[Create Order]â”€[Response] = <100ms each
Thread-2: [Create Order]â”€[Response]â”€[Create Order]â”€[Response]
Async Pool:
  Thread-A: [Send Emailâ”€â”€â”€2sâ”€â”€â”€]â”€[Send SMSâ”€â”€1sâ”€]
  Thread-B: [Send Emailâ”€â”€â”€2sâ”€â”€â”€]â”€[Send SMSâ”€â”€1sâ”€]

```

```
10 requests = 2 main threads + 2 async threads (efficient!)
```

When to Use @Async:

DO use for:

- Sending emails/SMS (non-critical)
- Logging to external systems
- Generating reports
- Processing large batches
- Calling slow third-party APIs (non-critical)

â€œDON'T use for:

- Critical business logic (payment processing)
- Operations requiring immediate feedback
- Database transactions (use carefully)
- Methods that return values to caller (use CompletableFuture)

Real Production Example:

```
@Service
public class UserRegistrationService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private NotificationAsyncService notificationService;

    @Transactional
    public UserResponse registerUser(UserRequest request) {
        // Critical: Save user (synchronous)
        User user = userRepository.save(new User(request));

        // Non-critical: Send welcome email (asynchronous)
        notificationService.sendWelcomeEmail(user);

        // Non-critical: Create user profile (asynchronous)
        notificationService.createUserProfile(user);

        return new UserResponse(user);
        // User gets instant response, notifications sent in background
    }
}

@Service
public class NotificationAsyncService {
```

```

@Async
public void sendWelcomeEmail(User user) {
    try {
        emailService.send(user.getEmail(), "Welcome!", "Thanks for
registering!");
        logger.info("Welcome email sent to: {}", user.getEmail());
    } catch (Exception e) {
        logger.error("Failed to send email: {}", e.getMessage());
        // Handle gracefully (don't crash main flow)
    }
}

@Async
public void createUserProfile(User user) {
    // Call external service to create profile
    profileService.createProfile(user);
}
}

```

Performance Impact:

- Without @Async: 3 seconds per registration
- With @Async: 100ms per registration
- **30x faster response time!**

¾ Transaction Management & @Transactional

Q10: "Explain @Transactional annotation and when to use it"

Answer: "@Transactional ensures database operations are **atomic** (all-or-nothing).

Real-World Analogy:

- You transfer \$100 from Account A to Account B
- If debit fails, credit should NOT happen
- If credit fails, debit should rollback

Without @Transactional (BAD):

```

@Service
public class BankingService {

    public void transferMoney(String fromAccount, String toAccount, BigDecimal
amount) {
        // Debit from Account A
        accountRepository.debit(fromAccount, amount); // Success

        // Something goes wrong here...
        if (amount.compareTo(BigDecimal.valueOf(10000)) > 0) {

```

```

        throw new RuntimeException("Amount too large!");
    }

    // Credit to Account B
    accountRepository.credit(toAccount, amount); // â€œ Never executes!

    // PROBLEM: Money debited but NOT credited (data inconsistency!)
}
}

```

Result: \$100 disappeared from Account A, never appeared in Account B!

With @Transactional (GOOD):

```

@Service
public class BankingService {

    @Transactional
    public void transferMoney(String fromAccount, String toAccount, BigDecimal
amount) {
        // Start transaction
        accountRepository.debit(fromAccount, amount);

        if (amount.compareTo(BigDecimal.valueOf(10000)) > 0) {
            throw new RuntimeException("Amount too large!");
        }

        accountRepository.credit(toAccount, amount);
        // Commit transaction (both operations succeed)
    }
}

```

If exception thrown:

- Both operations **rollback**
 - Database returns to original state
 - Data consistency maintained
-

@Transactional Configuration:

```

@Transactional(
    propagation = Propagation.REQUIRED, // Join existing transaction or create
new
    isolation = Isolation.READ_COMMITTED, // Isolation level
    timeout = 30, // Timeout after 30 seconds
    rollbackFor = Exception.class, // Rollback on any exception
    noRollbackFor = ValidationException.class // Don't rollback for validation
)

```

```
errors
)
public void complexOperation() {
    // Transaction managed automatically
}
```

Propagation Types:

Propagation	Behavior
REQUIRED (default)	Join existing transaction or create new
REQUIRES_NEW	Always create new transaction (suspend existing)
SUPPORTS	Use transaction if exists, non-transactional otherwise
NOT_SUPPORTED	Execute non-transactionally (suspend existing)
MANDATORY	Must be called within transaction (else throw exception)
NEVER	Must NOT be called within transaction

Example:

```
@Service
public class OrderService {

    @Transactional // Transaction A starts
    public void createOrder(OrderRequest request) {
        orderRepository.save(order);

        // Uses same transaction A
        orderItemService.saveOrderItems(order.getItems());

        // Creates NEW transaction B (independent)
        auditService.logOrderCreated(order);
    }
}

@Service
public class AuditService {

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logOrderCreated(Order order) {
        auditRepository.save(new AuditLog(order));
        // Even if Order transaction rolls back, audit log is saved!
    }
}
```

Isolation Levels:

Isolation	Read Uncommitted	Dirty Reads	Non-Repeatable Reads	Phantom Reads
READ_UNCOMMITTED		ââ	ââ	ââ
READ_COMMITTED			ââ	ââ
REPEATABLE_READ				ââ
SERIALIZABLE				

Production Best Practice:

```
@Transactional(isolation = Isolation.READ_COMMITTED) // Most common
public void processPayment() {
    // Balance between consistency and performance
}
```

When to Use @Transactional:

DO use for:

- Database write operations (INSERT, UPDATE, DELETE)
- Multiple operations that must succeed together
- Money transfers, inventory updates

ââ DON'T use for:

- Read-only operations (use `@Transactional(readonly = true)`)
- Long-running operations (blocks database connections)
- Methods calling external APIs (can't rollback external calls)

@Transactional with @Async (, DANGEROUS):

```
// ââ BAD: Both annotations on same method
@Async
@Transactional
public void processPayment(Payment payment) {
    // PROBLEM:
    // 1. @Async runs in separate thread
    // 2. @Transactional creates transaction in that thread
    // 3. Caller doesn't wait, can't handle exceptions
    // 4. Transaction might commit before exception thrown
    // 5. Unpredictable behavior!
}

// GOOD: Separate concerns
@Service
public class PaymentService {
```



```
@Autowired
private PaymentTransactionService transactionService;

@Async
public void processPayment(Payment payment) {
    // Async processing
    transactionService.savePayment(payment);
}

@Service
public class PaymentTransactionService {
    @Transactional
    public void savePayment(Payment payment) {
        paymentRepository.save(payment);
    }
}
```

Real Production Example:

```
@Service
public class OrderService {

    @Transactional
    public OrderResponse createOrder(OrderRequest request) {
        // 1. Validate inventory (read)
        Product product = productRepository.findById(request.getProductId())
            .orElseThrow(() -> new ProductNotFoundException());

        if (product.getStock() < request.getQuantity()) {
            throw new InsufficientStockException();
        }

        // 2. Create order (write)
        Order order = new Order(request);
        orderRepository.save(order);

        // 3. Update inventory (write)
        product.setStock(product.getStock() - request.getQuantity());
        productRepository.save(product);

        // 4. Create order items (write)
        OrderItem item = new OrderItem(order, product, request.getQuantity());
        orderItemRepository.save(item);

        // All 4 operations succeed together or rollback together
        return new OrderResponse(order);
    }
}
```

Without @Transactional:

- Order created
- Inventory update fails ❌
- Result: Order exists but inventory NOT updated (oversold!)

With @Transactional:

- Order creation rolled back
- Inventory unchanged
- Result: Data consistency maintained"

Self-Invocation Problem

Q11: "What is the self-invocation problem with @Async and @Transactional?"

Answer: "Self-invocation" is when a method calls another method **in the same class** that has @Async or @Transactional.

The Problem: Spring uses **proxies** for @Async and @Transactional. When you call a method from within the same class, the **proxy is bypassed**, so the annotation doesn't work!

Example 1: @Async Self-Invocation (BROKEN)

```
@Service
public class NotificationService {

    public void processOrder(Order order) {
        saveOrder(order);
        sendEmail(order); // ❌ @Async doesn't work!
    }

    @Async
    public void sendEmail(Order order) {
        // Expected: Runs in separate thread
        // Reality: Runs in SAME thread (synchronously)
        emailService.send(order.getCustomerEmail(), "Order Confirmed");
    }
}
```

Why it doesn't work:

```
Client  Spring Proxy (@Async logic)  NotificationService.processOrder()
      â†“
      processOrder() calls sendEmail() DIRECTLY
      â†“
      Bypasses proxy ❌
```

```

        â†“
        @Async ignored, runs synchronously

```

Solution 1: Separate Service (RECOMMENDED)

```

@Service
public class OrderProcessingService {
    @Autowired
    private EmailAsyncService emailService; // Separate service

    public void processOrder(Order order) {
        saveOrder(order);
        emailService.sendEmail(order); // Works! Goes through proxy
    }
}

@Service
public class EmailAsyncService {

    @Async
    public void sendEmail(Order order) {
        // Now runs in separate thread
        emailService.send(order.getCustomerEmail(), "Order Confirmed");
    }
}

```

Now it works:

```

OrderProcessingService Spring Proxy (@Async logic) EmailAsyncService.sendEmail()
        â†“
        Proxy intercepts call
        â†“
        Executes in async thread pool

```

Example 2: @Transactional Self-Invocation (BROKEN)

```

@Service
public class PaymentService {

    public void processPayment(Payment payment) {
        validatePayment(payment);
        savePayment(payment); // âœ“ @Transactional doesn't work!
    }

    @Transactional

```

```
    public void savePayment(Payment payment) {  
        // Expected: Runs in transaction  
        // Reality: NO transaction started!  
        paymentRepository.save(payment);  
    }  
}
```

If exception thrown: No rollback! Data inconsistency!

Solution: Separate Service

```
@Service  
public class PaymentService {  
    @Autowired  
    private PaymentTransactionService transactionService;  
  
    public void processPayment(Payment payment) {  
        validatePayment(payment);  
        transactionService.savePayment(payment); // Works!  
    }  
}  
  
@Service  
public class PaymentTransactionService {  
  
    @Transactional  
    public void savePayment(Payment payment) {  
        paymentRepository.save(payment);  
        // Transaction works  
    }  
}
```

☐ SPECIAL CASE: Both Methods Have @Async

```
@Service  
public class NotificationService {  
  
    @Async // Method 1 has @Async  
    public void processNotifications(Order order) {  
        sendEmail(order); // Calling Method 2  
    }  
  
    @Async // Method 2 also has @Async  
    public void sendEmail(Order order) {  
        emailService.send(order);  
    }  
}
```

What happens:

1. `processNotifications()` runs in async thread (called from outside)
2. Inside `processNotifications()`, calls `sendEmail()` directly
3. `sendEmail()`'s `@Async` is **IGNORED** ☹️
4. `sendEmail()` runs in the **SAME async thread**, NOT a new thread

Key Point:

- `processNotifications()` is async (because called from outside class)
- ☹️ `sendEmail()` is NOT async (because called from inside same class)
- Both methods run in the **same thread**

☹️ SPECIAL CASE: Both Methods Have @Transactional

```
@Service
public class OrderService {

    @Transactional
    public void createOrder(OrderRequest request) {
        Order order = new Order(request);
        saveOrder(order); // Calling another @Transactional method
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void saveOrder(Order order) {
        orderRepository.save(order);
    }
}
```

What happens:

1. `createOrder()` starts Transaction A
2. Inside `createOrder()`, calls `saveOrder()` directly
3. `saveOrder()`'s `@Transactional` is **IGNORED** ☹️
4. `saveOrder()` runs in **Transaction A** (not a new transaction)
5. `REQUIRES_NEW` propagation is ignored

Expected: `saveOrder()` creates new transaction (independent) **Reality:** `saveOrder()` uses same transaction as `createOrder()`

How to Detect Self-Invocation in Code Reviews:

☹️ RED FLAGS (Self-Invocation):

- `@Async` or `@Transactional` method called from same class
- `this.methodName()` where `methodName` has `@Async/@Transactional`
- No `@Autowired` dependency for async/transactional operations

GREEN FLAGS (Correct):

- @Async/@Transactional in separate @Service class
- @Autowired dependency injected
- Method called via injected dependency

Complete Example - E-Commerce Order Flow:

```
// ❌ BAD: Everything in one service (self-invocation)
@Service
public class BadOrderService {

    public OrderResponse createOrder(OrderRequest request) {
        validateOrder(request); // Normal method
        saveOrder(request);      // ❌ @Transactional ignored!
        sendNotifications();     // ❌ @Async ignored!
        updateInventory();       // ❌ @Async ignored!
        return new OrderResponse();
    }

    @Transactional
    public void saveOrder(OrderRequest request) {
        orderRepository.save(new Order(request));
    }

    @Async
    public void sendNotifications() {
        emailService.send("Order confirmed");
    }

    @Async
    public void updateInventory() {
        inventoryService.update();
    }
}

// GOOD: Separated services (no self-invocation)
@Service
public class OrderService {
    @Autowired
    private OrderTransactionService transactionService;
    @Autowired
    private NotificationAsyncService notificationService;
    @Autowired
    private InventoryAsyncService inventoryService;

    public OrderResponse createOrder(OrderRequest request) {
        validateOrder(request);

        // All these go through Spring proxies
        Order order = transactionService.saveOrder(request);
    }
}
```

```
        notificationService.sendNotifications(order);
        inventoryService.updateInventory(order);

        return new OrderResponse(order);
    }
}

@Service
public class OrderTransactionService {
    @Transactional
    public Order saveOrder(OrderRequest request) {
        return orderRepository.save(new Order(request));
    }
}

@Service
public class NotificationAsyncService {
    @Async
    public void sendNotifications(Order order) {
        emailService.send("Order confirmed");
    }
}

@Service
public class InventoryAsyncService {
    @Async
    public void updateInventory(Order order) {
        inventoryService.update();
    }
}
```

Benefits of separation:

- @Async and @Transactional work correctly
- Single Responsibility Principle
- Easy to test each service independently
- No proxy bypass issues

Summary:

Scenario	Works?	Reason
Method A (no annotation) calls Method B (@Async) in same class	❌	Proxy bypassed
Method A (no annotation) calls Method B (@Async) in different class		Goes through proxy
Method A (@Async) calls Method B (@Async) in same class	⚠️ Partial	A is async, B is not
Method A (@Async) calls Method B (@Async) in different class		Both async

Scenario	Works?	Reason
Method A (@Transactional) calls Method B (@Transactional REQUIRES_NEW) in same class	❌	B's settings ignored
Method A (@Transactional) calls Method B (@Transactional REQUIRES_NEW) in different class		B creates new transaction

Golden Rule: Never call @Async or @Transactional methods from the same class. Always separate into different services!"

Database Design - Views & Stored Procedures

Q7: "Explain your database schema design with views and stored procedures"

Answer: "I designed a multi-schema PostgreSQL database with **5 separate schemas** for microservices isolation, using views for data abstraction and stored procedures for complex operations:

1. Schema Architecture:

```
-- Separate schemas per microservice
CREATE SCHEMA IF NOT EXISTS evaluation;
CREATE SCHEMA IF NOT EXISTS sampling;
CREATE SCHEMA IF NOT EXISTS evidence;
CREATE SCHEMA IF NOT EXISTS remediation;
CREATE SCHEMA IF NOT EXISTS jira_service;
```

Benefits:

- **Isolation:** Each service has its own namespace
- **Security:** Grant permissions per schema
- **No name collisions:** Can have `evaluation.users` and `jira_service.users`
- **Clear ownership:** Schema = bounded context

2. Core Tables with Indexes:

```
-- Evaluation service - User table
CREATE TABLE evaluation.users (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(100) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  age INTEGER,
  department VARCHAR(100),
  salary DECIMAL(12, 2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



```
-- Strategic indexes for performance
CREATE INDEX idx_users_username ON evaluation.users(username);
CREATE INDEX idx_users_email ON evaluation.users(email);
CREATE INDEX idx_users_department ON evaluation.users(department);
```

Why these indexes?

- **username** & **email**: Frequent lookups, authentication
- **department**: Analytics queries (GROUP BY department)

3. Creating Materialized Views for Analytics:

```
-- POSITIVE: Aggregate view for department statistics
CREATE MATERIALIZED VIEW evaluation.department_stats AS
SELECT
    department,
    COUNT(*) as employee_count,
    AVG(salary) as avg_salary,
    MIN(salary) as min_salary,
    MAX(salary) as max_salary,
    SUM(salary) as total_salary_cost
FROM evaluation.users
WHERE department IS NOT NULL
GROUP BY department;

-- Create index on materialized view
CREATE INDEX idx_dept_stats_department ON evaluation.department_stats(department);

-- Refresh strategy
REFRESH MATERIALIZED VIEW CONCURRENTLY evaluation.department_stats;
```

Benefits:

- Pre-computed aggregations (fast queries)
- No JOIN overhead at query time
- Can refresh on schedule (hourly/daily)
- CONCURRENTLY allows reads during refresh

4. Regular Views for Data Abstraction:

```
-- POSITIVE: Security view - hide sensitive data
CREATE VIEW evaluation.users_public AS
SELECT
    id,
    username,
    first_name,
    last_name,
    department,
```

```

        created_at
    FROM evaluation.users;

-- Grant access to read-only role
GRANT SELECT ON evaluation.users_public TO readonly_user;

```

5. View for Cross-Service Data:

```

-- POSITIVE: Join users with JIRA tickets (if needed)
CREATE VIEW analytics.user_tickets AS
SELECT
    u.id,
    u.username,
    u.email,
    u.department,
    j.ticket_key,
    j.summary,
    j.priority,
    j.status,
    j.created_at as ticket_created_at
FROM evaluation.users u
LEFT JOIN jira_service.jira_tickets j ON u.username = j.assignee;

```

6. Stored Procedure for Complex Business Logic:

```

-- POSITIVE: Bulk user creation with validation
CREATE OR REPLACE FUNCTION evaluation.create_user_batch(
    p_users JSONB
) RETURNS TABLE(user_id BIGINT, username VARCHAR, status VARCHAR) AS $$
DECLARE
    user_record JSONB;
    new_user_id BIGINT;
BEGIN
    -- Loop through JSON array
    FOR user_record IN SELECT * FROM jsonb_array_elements(p_users)
    LOOP
        BEGIN
            -- Validate email format
            IF user_record->>'email' !~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}$' THEN
                RETURN QUERY SELECT NULL::BIGINT, user_record->>'username',
                'INVALID_EMAIL';
                CONTINUE;
            END IF;

            -- Check for duplicates
            IF EXISTS(SELECT 1 FROM evaluation.users WHERE username = user_record->>'username') THEN
                RETURN QUERY SELECT NULL::BIGINT, user_record->>'username',

```

```

'DUPLICATE';
        CONTINUE;
    END IF;

    -- Insert user
    INSERT INTO evaluation.users (username, email, first_name, last_name,
department, salary)
    VALUES (
        user_record->>'username',
        user_record->>'email',
        user_record->>'first_name',
        user_record->>'last_name',
        user_record->>'department',
        (user_record->>'salary')::DECIMAL
    )
    RETURNING id INTO new_user_id;

    RETURN QUERY SELECT new_user_id, user_record->>'username', 'SUCCESS';

    EXCEPTION WHEN OTHERS THEN
        RETURN QUERY SELECT NULL::BIGINT, user_record->>'username', 'ERROR: '
|| SQLERRM;
    END;
END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Call from application
SELECT * FROM evaluation.create_user_batch('[
    {"username": "john.doe", "email": "john@example.com", "first_name": "John",
"last_name": "Doe"},
    {"username": "jane.smith", "email": "jane@example.com", "first_name": "Jane",
"last_name": "Smith"}
]':::JSONB);

```

Benefits:

- **Atomic operations:** All or nothing
- **Validation in database:** Email format, duplicates
- **Error handling:** Returns status per user
- **Performance:** Single round-trip, bulk insert

7. Stored Procedure for Data Archiving:

```

-- POSITIVE: Archive old remediation tasks
CREATE OR REPLACE PROCEDURE remediation.archive_completed_tasks(
    p_days_old INTEGER DEFAULT 90
) AS $$
DECLARE
    v_archived_count INTEGER;
BEGIN

```

```

-- Create archive table if not exists
CREATE TABLE IF NOT EXISTS remediation.remediation_tasks_archive (
    LIKE remediation.remediation_tasks INCLUDING ALL
);

-- Move completed tasks older than N days
WITH moved_rows AS (
    DELETE FROM remediation.remediation_tasks
    WHERE status = 'COMPLETED'
    AND completed_at < CURRENT_DATE - p_days_old
    RETURNING *
)
INSERT INTO remediation.remediation_tasks_archive
SELECT * FROM moved_rows;

GET DIAGNOSTICS v_archived_count = ROW_COUNT;

RAISE NOTICE 'Archived % completed tasks older than % days', v_archived_count,
p_days_old;
END;
$$ LANGUAGE plpgsql;

-- Schedule to run monthly
CALL remediation.archive_completed_tasks(90);

```

8. Trigger Function for Auto-Update Timestamp:

```

-- POSITIVE: Auto-update updated_at column
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Apply to all tables with updated_at
CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON evaluation.users
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

CREATE TRIGGER update_remediation_tasks_updated_at
BEFORE UPDATE ON remediation.remediation_tasks
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

```

Benefits:

- No application logic needed
- Consistent behavior
- Automatic audit trail

9. Stored Function for Business Calculations:

```
-- POSITIVE: Calculate user risk score
CREATE OR REPLACE FUNCTION evaluation.calculate_risk_score(
    p_username VARCHAR
) RETURNS INTEGER AS $$
DECLARE
    v_risk_score INTEGER := 0;
    v_age INTEGER;
    v_email VARCHAR;
BEGIN
    -- Get user details
    SELECT age, email INTO v_age, v_email
    FROM evaluation.users
    WHERE username = p_username;

    -- Risk factors
    IF v_email LIKE '%@tempmail.%' OR v_email LIKE '%@guerrillamail.%' THEN
        v_risk_score := v_risk_score + 50; -- Suspicious email
    END IF;

    IF v_email ~ '\d{5,}' THEN
        v_risk_score := v_risk_score + 30; -- Too many numbers
    END IF;

    IF LENGTH(p_username) < 5 THEN
        v_risk_score := v_risk_score + 20; -- Short username
    END IF;

    RETURN v_risk_score;
END;
$$ LANGUAGE plpgsql;

-- Usage
SELECT username, evaluation.calculate_risk_score(username) as risk_score
FROM evaluation.users
WHERE evaluation.calculate_risk_score(username) > 50;
```

Real Benefits in Production:

- Materialized views: **10x faster** dashboard queries
- Stored procedures: **50% less application code**
- Triggers: **Zero bugs** for updated_at
- Schema isolation: **Easy to scale** per service

Performance Metrics:

- Before materialized view: 5s query time
- After materialized view: 50ms query time
- Bulk insert stored procedure: **2000 users/sec**

Q8: "What are the POSITIVE and NEGATIVE scenarios for using Views vs Stored Procedures?"

Answer:

****VIEWS - Positive Scenarios ****

1. Data Security & Access Control:

```
-- Hide salary from regular users
CREATE VIEW evaluation.users_basic AS
SELECT id, username, email, first_name, last_name, department
FROM evaluation.users;

GRANT SELECT ON evaluation.users_basic TO app_user;
REVOKE ALL ON evaluation.users FROM app_user;
```

2. Simplify Complex Joins:

```
-- Application doesn't need to know JOIN logic
CREATE VIEW analytics.user_activity AS
SELECT
    u.username,
    COUNT(DISTINCT j.ticket_key) as total_tickets,
    COUNT(DISTINCT r.task_id) as total_tasks,
    AVG(CASE WHEN j.priority = 'CRITICAL' THEN 4
            WHEN j.priority = 'HIGH' THEN 3
            ELSE 2 END) as avg_priority
FROM evaluation.users u
LEFT JOIN jira_service.jira_tickets j ON u.username = j.assignee
LEFT JOIN remediation.remediation_tasks r ON u.username = r.assigned_to
GROUP BY u.username;

-- Simple query in application
SELECT * FROM analytics.user_activity WHERE total_tickets > 10;
```

3. Backward Compatibility After Schema Change:

```
-- Rename column without breaking old applications
ALTER TABLE evaluation.users RENAME COLUMN phone_number TO mobile_phone;

-- Old applications still work
CREATE VIEW evaluation.users_legacy AS
SELECT
    id,
    username,
    mobile_phone as phone_number, -- Map new name to old name
    email
FROM evaluation.users;
```

4. Aggregated Data for Analytics:

```
-- Pre-defined metrics
CREATE MATERIALIZED VIEW analytics.daily_user_signups AS
SELECT
    DATE(created_at) as signup_date,
    COUNT(*) as new_users,
    COUNT(DISTINCT department) as unique_departments
FROM evaluation.users
GROUP BY DATE(created_at);
```

VIEWS - Negative Scenarios ⚠️

1. Performance Problem - View on View:

```
-- ⚠️ BAD: Nested views kill performance
CREATE VIEW evaluation.active_users AS
SELECT * FROM evaluation.users WHERE is_active = true;

CREATE VIEW evaluation.engineering_users AS
SELECT * FROM evaluation.active_users WHERE department = 'Engineering';

CREATE VIEW evaluation.senior_engineering AS
SELECT * FROM evaluation.engineering_users WHERE salary > 100000;

-- This query is SLOW - 3 layers of views!
SELECT * FROM evaluation.senior_engineering;

-- BETTER: Single view or direct query
CREATE VIEW evaluation.senior_engineering_direct AS
SELECT * FROM evaluation.users
WHERE is_active = true
AND department = 'Engineering'
AND salary > 100000;
```

2. UPDATE/DELETE on Views Can Be Tricky:

```
-- ⚠️ Cannot update through view with JOIN
CREATE VIEW analytics.user_with_dept_stats AS
SELECT u.*, d.avg_salary
FROM evaluation.users u
JOIN evaluation.department_stats d ON u.department = d.department;

-- ⚠️ This fails!
UPDATE analytics.user_with_dept_stats SET first_name = 'John' WHERE id = 1;
-- ERROR: cannot update view with joins
```

```
-- BETTER: Use INSTEAD OF trigger
CREATE OR REPLACE FUNCTION update_user_view()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE evaluation.users
    SET first_name = NEW.first_name
    WHERE id = NEW.id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_user_via_view
INSTEAD OF UPDATE ON analytics.user_with_dept_stats
FOR EACH ROW EXECUTE FUNCTION update_user_view();
```

3. Materialized View Staleness:

```
-- Data can be outdated
CREATE MATERIALIZED VIEW evaluation.department_stats AS
SELECT department, COUNT(*) as count FROM evaluation.users GROUP BY department;

-- New user inserted...
INSERT INTO evaluation.users (username, department) VALUES ('test', 'IT');

-- View still shows old count!
SELECT * FROM evaluation.department_stats; -- Stale data

-- SOLUTION: Refresh schedule
REFRESH MATERIALIZED VIEW CONCURRENTLY evaluation.department_stats;

-- Or use regular view (always fresh but slower)
CREATE VIEW evaluation.department_stats_live AS
SELECT department, COUNT(*) as count FROM evaluation.users GROUP BY department;
```

****STORED PROCEDURES - Positive Scenarios ****

1. Complex Multi-Step Business Logic:

```
-- POSITIVE: User onboarding workflow
CREATE OR REPLACE PROCEDURE evaluation.onboard_new_employee(
    p_username VARCHAR,
    p_email VARCHAR,
    p_department VARCHAR,
    p_salary DECIMAL
) AS $$
DECLARE
    v_user_id BIGINT;
BEGIN
    -- Step 1: Create user
```



```

INSERT INTO evaluation.users (username, email, department, salary)
VALUES (p_username, p_email, p_department, p_salary)
RETURNING id INTO v_user_id;

-- Step 2: Create welcome JIRA ticket
INSERT INTO jira_service.jira_tickets (ticket_key, summary, assignee, status)
VALUES (
    'ONBOARD-' || v_user_id,
    'Complete onboarding for ' || p_username,
    'hr.manager',
    'TO_DO'
);

-- Step 3: Create initial remediation task (setup laptop)
INSERT INTO remediation.remediation_tasks (task_id, description, assigned_to)
VALUES (
    'SETUP-' || v_user_id,
    'Setup laptop and access for ' || p_username,
    'it.admin'
);

COMMIT;
END;
$$ LANGUAGE plpgsql;

```

Benefits:

- Atomic: All steps succeed or all rollback
- Reusable: One call from any application
- Business logic in database (single source of truth)

2. Batch Operations with Error Handling:

```

-- POSITIVE: Bulk update with detailed logging
CREATE OR REPLACE PROCEDURE evaluation.bulk_salary_increase(
    p_department VARCHAR,
    p_percentage DECIMAL,
    OUT p_updated_count INTEGER,
    OUT p_failed_count INTEGER
) AS $$
DECLARE
    user_rec RECORD;
BEGIN
    p_updated_count := 0;
    p_failed_count := 0;

    FOR user_rec IN
        SELECT id, username, salary FROM evaluation.users WHERE department =
        p_department
    LOOP
        BEGIN

```

```

        UPDATE evaluation.users
        SET salary = salary * (1 + p_percentage / 100)
        WHERE id = user_rec.id;

        p_updated_count := p_updated_count + 1;

    EXCEPTION WHEN OTHERS THEN
        p_failed_count := p_failed_count + 1;
        RAISE NOTICE 'Failed to update %: %', user_rec.username, SQLERRM;
    END;
END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Call it
CALL evaluation.bulk_salary_increase('Engineering', 10, NULL, NULL);

```

3. Scheduled Maintenance Tasks:

```

-- POSITIVE: Cleanup old data
CREATE OR REPLACE PROCEDURE maintenance.cleanup_old_logs(
    p_retention_days INTEGER DEFAULT 30
) AS $$
DECLARE
    v_deleted_count INTEGER;
BEGIN
    -- Delete old evidence records
    DELETE FROM evidence.evidence_records
    WHERE created_at < CURRENT_DATE - p_retention_days;
    GET DIAGNOSTICS v_deleted_count = ROW_COUNT;
    RAISE NOTICE 'Deleted % old evidence records', v_deleted_count;

    -- Delete old jira tickets
    DELETE FROM jira_service.jira_tickets
    WHERE status = 'DONE' AND updated_at < CURRENT_DATE - p_retention_days;
    GET DIAGNOSTICS v_deleted_count = ROW_COUNT;
    RAISE NOTICE 'Deleted % old jira tickets', v_deleted_count;

    -- Vacuum tables
    VACUUM ANALYZE evidence.evidence_records;
    VACUUM ANALYZE jira_service.jira_tickets;
END;
$$ LANGUAGE plpgsql;

```

STORED PROCEDURES - Negative Scenarios ☒

1. Performance Problem - Row-by-Row Processing:

```
-- ⚠️ BAD: Looping through millions of rows
CREATE OR REPLACE PROCEDURE evaluation.update_all_salaries_slow() AS $$
DECLARE
    user_rec RECORD;
BEGIN
    FOR user_rec IN SELECT id, salary FROM evaluation.users
    LOOP
        UPDATE evaluation.users
        SET salary = salary * 1.05
        WHERE id = user_rec.id; -- One update per row!
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- ⚠️ For 1 million users: takes 30 minutes!

-- BETTER: Set-based operation
CREATE OR REPLACE PROCEDURE evaluation.update_all_salaries_fast() AS $$
BEGIN
    UPDATE evaluation.users
    SET salary = salary * 1.05; -- Single UPDATE for all rows
END;
$$ LANGUAGE plpgsql;

-- For 1 million users: takes 5 seconds!
```

2. Lock Contention:

```
-- ⚠️ BAD: Long-running procedure locks table
CREATE OR REPLACE PROCEDURE evaluation.process_all_users() AS $$
DECLARE
    user_rec RECORD;
BEGIN
    -- This locks the entire table!
    FOR user_rec IN SELECT * FROM evaluation.users FOR UPDATE
    LOOP
        -- Expensive operation (API call, sleep, etc.)
        PERFORM pg_sleep(0.1); -- Simulated delay

        UPDATE evaluation.users SET processed = true WHERE id = user_rec.id;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- ⚠️ Other transactions blocked for minutes!

-- BETTER: Process in small batches
CREATE OR REPLACE PROCEDURE evaluation.process_users_batch(
    p_batch_size INTEGER DEFAULT 100
) AS $$
DECLARE
```

```

    v_processed INTEGER;
BEGIN
    LOOP
        UPDATE evaluation.users
        SET processed = true
        WHERE id IN (
            SELECT id FROM evaluation.users
            WHERE processed = false
            LIMIT p_batch_size
        );

        GET DIAGNOSTICS v_processed = ROW_COUNT;
        EXIT WHEN v_processed = 0;

        -- Release locks between batches
        COMMIT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

3. Hidden Business Logic (Maintenance Nightmare):

```

-- ❌ BAD: Complex logic hidden in database
CREATE OR REPLACE PROCEDURE evaluation.calculate_bonuses() AS $$
BEGIN
    -- 500 lines of complex business logic...
    -- Hard to test
    -- Hard to version control
    -- Hard to debug
    -- No one knows what it does!
END;
$$ LANGUAGE plpgsql;

-- BETTER: Keep complex logic in application
-- Use stored procedures only for:
-- - Data validation
-- - Atomicity requirements
-- - Performance-critical operations
```

Decision Matrix:

Scenario	Use View	Use Stored Procedure
Read-only data abstraction	Yes	❌ No
Hide sensitive columns	Yes	❌ No
Complex aggregations (refreshed periodically)	Materialized View	❌ No

Scenario	Use View	Use Stored Procedure
Multi-table updates (atomic)	☐ No	Yes
Business workflows	☐ No	Yes
Data validation & constraints	☑ Maybe	Yes
Frequently changing logic	☐ No (application code better)	☑ Maybe

My Experience:

- Used **views** for: Analytics dashboards, read-only APIs, security (11 views created)
- Used **stored procedures** for: Bulk operations, data archiving, cleanup tasks (7 procedures)
- **Avoided** stored procedures for complex business logic (keep in application)
- Result: **30% faster queries, cleaner application code**

Q9: "How do you handle secrets in AWS?"

Answer: "I use **AWS Secrets Manager** integrated with Terraform:

Terraform Configuration:

```
# Create secret
resource "aws_secretsmanager_secret" "db_password" {
  name = "microservices/dev/db-password"
  description = "Database password for microservices"
}

# Store secret value
resource "aws_secretsmanager_secret_version" "db_password" {
  secret_id = aws_secretsmanager_secret.db_password.id
  secret_string = jsonencode({
    username = var.db_username
    password = var.db_password
    host      = aws_db_instance.main.address
    port      = 5432
    dbname    = var.db_name
  })
}
```

ECS Task Definition:

```
secrets = [{
  name      = "SPRING_DATASOURCE_PASSWORD"
  valueFrom = aws_secretsmanager_secret.db_password.arn
}]
```

Spring Boot Access:

```
spring:
  datasource:
    url: \${DB_URL}
    username: \${DB_USERNAME}
    password: \${DB_PASSWORD} # Injected from Secrets Manager
```

Best Practices:

1. **Never commit secrets to Git**
2. **Use environment variables** for local dev
3. **Rotate secrets regularly** (Secrets Manager auto-rotation)
4. **IAM policies** restrict access
5. **Audit logging** with CloudTrail

Local Development:

```
# Use environment variables
export DB_PASSWORD=\"devpassword\"

# Or AWS CLI
aws secretsmanager get-secret-value --secret-id db-password
```

Lambda Access:

```
import boto3
secretsmanager = boto3.client('secretsmanager')
response = secretsmanager.get_secret_value(SecretId='db-password')
secret = json.loads(response['SecretString'])
```

My implementation:

- JWT secret in Secrets Manager
- Database passwords in Secrets Manager
- API keys in Parameter Store
- Never hardcoded in code or Terraform files"

Microservices Questions

Q8: "How do you implement inter-service communication?"

Answer: "I use both **synchronous** and **asynchronous** communication:

1. Synchronous (REST APIs):

```

@Service
public class EvaluationService {
    @Autowired
    private RestTemplate restTemplate;

    public SamplingData getSamplingData(Long id) {
        String url = "http://sampling-service:8082/api/samples/" + id;
        return restTemplate.getForObject(url, SamplingData.class);
    }
}

```

2. Asynchronous (Kafka):

```

@Service
public class EventPublisher {
    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    public void publishEvaluationCompleted(Long id) {
        kafkaTemplate.send("evaluation-completed", id.toString());
    }
}

@Service
public class RemediationService {
    @KafkaListener(topics = "evaluation-completed")
    public void handleEvaluationCompleted(String message) {
        // Process remediation
    }
}

```

3. Service Discovery (AWS):

- ALB routes by path: `/evaluation-service/*`
- ECS Service Connect for internal communication
- DNS-based (service names resolve to IPs)

4. Circuit Breaker (Resilience4j):

```

@CircuitBreaker(name = "samplingService", fallbackMethod =
    "getSamplingDataFallback")
public SamplingData getSamplingData(Long id) {
    return restTemplate.getForObject(url, SamplingData.class);
}

public SamplingData getSamplingDataFallback(Long id, Exception ex) {

```

```
        return SamplingData.empty();
    }
```

When to use each:

- **REST:** Real-time, request-response (user-facing)
- **Kafka:** Async, event-driven, high throughput
- **Step Functions:** Complex workflows, error handling

My implementation:

- REST for immediate responses
- Kafka for notifications
- Step Functions for orchestration (Sampling Evaluation Remediation)"

Q9: "How do you implement CompletableFuture for async operations?"

Answer: "I implement separate thread pools for I/O and CPU tasks:

Configuration:

```
@Configuration
public class AsyncExecutorConfig {
    @Bean(name = \"ioTaskExecutor\")
    public Executor ioTaskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(20);    // I/O can handle more threads
        executor.setMaxPoolSize(50);
        executor.setQueueCapacity(500);
        executor.setThreadNamePrefix(\"IO-\");
        executor.setRejectedPolicy(new CallerRunsPolicy());
        executor.initialize();
        return executor;
    }

    @Bean(name = \"cpuTaskExecutor\")
    public Executor cpuTaskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(Runtime.getRuntime().availableProcessors());
        executor.setMaxPoolSize(Runtime.getRuntime().availableProcessors() * 2);
        return executor;
    }
}
```

Service Implementation:

```
@Service
public class AsyncUserService {
```



```

@Autowired
@Qualifier("ioTaskExecutor")
private Executor ioExecutor;

@Autowired
@Qualifier("cpuTaskExecutor")
private Executor cpuExecutor;

// I/O-bound operation (database, API calls)
public CompletableFuture<List<User>> getAllUsersAsync() {
    return CompletableFuture.supplyAsync(() -> {
        return userRepository.findAll(); // Database I/O
    }, ioExecutor);
}

// CPU-bound operation (data processing)
public CompletableFuture<List<UserDTO>> processUsersToDTO(List<User> users) {
    return CompletableFuture.supplyAsync(() -> {
        return users.stream()
            .map(this::calculateBonus) // Heavy computation
            .collect(Collectors.toList());
    }, cpuExecutor);
}

// Combined workflow
public CompletableFuture<List<UserDTO>> getAllUsersWithProcessing() {
    return getAllUsersAsync() // I/O executor
        .thenComposeAsync(users -> processUsersToDTO(users), cpuExecutor); //
CPU executor
}
}

```

Controller:

```

@GetMapping("/api/async/users")
public CompletableFuture<ResponseEntity<List<UserDTO>>> getUsers() {
    return asyncUserService.getAllUsersWithProcessing()
        .thenApply(ResponseEntity::ok)
        .exceptionally(ex -> ResponseEntity.status(500).build());
}

```

Benefits:

- Non-blocking I/O
- CPU tasks don't block I/O threads
- Better resource utilization
- Handles high concurrency

In my project, this improved throughput by 300% under load testing."

Scenario-Based Questions

Q10: "A user reports 401 Unauthorized. How do you troubleshoot?"

Answer: Step 1: Check logs (CloudWatch in AWS)

```
2026-01-13 10:30:45 - JWT token is expired
2026-01-13 10:30:45 - Unauthorized error: Full authentication required
```

Step 2: Verify token format

```
# Should be: Authorization: Bearer <token>
curl -H \"Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...\"
http://localhost:8081/api/users
```

Step 3: Validate token manually

```
# Test login
curl -X POST http://localhost:8081/api/auth/login \\
  -H \"Content-Type: application/json\" \\
  -d '{\"username\":\"admin\",\"password\":\"admin123\"}'

# Validate token
curl -H \"Authorization: Bearer <token>\" http://localhost:8081/api/auth/validate
```

Step 4: Check SecurityConfig

```
// Ensure endpoint isn't blocked
.requestMatchers(\"/api/users/**\").hasAnyRole(\"USER\", \"ADMIN\")
```

Common Issues:

1. **Token expired** User needs to re-login
2. **Wrong secret key** Token signed with different key
3. **Missing 'Bearer ' prefix** Fix: **Bearer <token>**
4. **User doesn't have required role** 403 Forbidden (not 401)
5. **CORS issue** Pre-flight OPTIONS request fails

My Solution:

- Added detailed logging in JwtAuthenticationFilter
- Created /api/auth/validate endpoint for debugging
- Clear error messages in JwtAuthenticationEntryPoint

Q11: "How do you handle database connection pool exhaustion?"

Answer: Scenario: Users see 'Connection timeout' errors

Step 1: Identify the issue

```
HikariPool-1 - Connection is not available, request timed out after 30000ms
```

Step 2: Check metrics (CloudWatch RDS)

- DatabaseConnections: 100 (max_connections)
- Active connections: 98
- Idle connections: 2

Step 3: Analyze code for connection leaks

```
// BAD: Connection leak
public List<User> getUsers() {
    Connection conn = dataSource.getConnection();
    // ... query ...
    // FORGOT TO CLOSE!
}

// GOOD: Auto-close
public List<User> getUsers() {
    try (Connection conn = dataSource.getConnection()) {
        // ... query ...
    } // Auto-closed
}

// BEST: Use JPA/Spring Data
public List<User> getUsers() {
    return userRepository.findAll(); // Spring manages connections
}
```

Step 4: Tune HikariCP

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 50 # Increase from default 10
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000
      leak-detection-threshold: 60000 # Detect leaks
```

Step 5: Scale RDS

```
# Terraform: Increase database connections
resource "aws_db_instance" "main" {
    instance_class = "db.t3.medium" # More memory = more connections

    parameter_group_name = aws_db_parameter_group.main.name
}

resource "aws_db_parameter_group" "main" {
    parameter {
        name = "max_connections"
        value = "200" # Increase from 100
    }
}
```

Root Cause in my experience:

- N+1 query problem (LazyInitializationException)
- Long-running transactions blocking connections
- Not closing ResultSets/Statements
- Connection pool too small for traffic

Solution I implemented:

- Enabled leak detection
- Added connection pool monitoring
- Used @Transactional properly
- Scaled RDS instance"

Success Stories

Success Story 1: "How did you improve API performance?"

Situation: API endpoint `/api/users` took 5 seconds to return 10,000 users.

Task: Reduce response time to <500ms.

Action:

1. Implemented pagination

```
@GetMapping("/api/users")
public Page<UserDTO> getUsers(Pageable pageable) {
    return userRepository.findAll(pageable)
        .map(userMapper::toDTO);
}

// Request: /api/users?page=0&size=20
```

2. Added database indexes

```
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_created_at ON users(created_at);
```

3. Used projections (not full entities)

```
@Query(\"SELECT new UserDTO(u.id, u.username, u.email) FROM User u\")  
List<UserDTO> findAllUsernames();
```

4. Implemented caching

```
@Cacheable(\"users\")  
public List<User> findAll() { ... }
```

5. CompletableFuture for parallel operations

```
CompletableFuture<List<User>> users = getAllUsersAsync();  
CompletableFuture<List<Order>> orders = getOrdersAsync();  
  
CompletableFuture.allOf(users, orders).join();
```

Result:

- Response time: 5s 200ms (96% improvement)
- Database queries: 10,000 20 (pagination)
- Memory usage: 500MB 50MB
- Throughput: 10 req/s 500 req/s

Learnings:

- Always paginate large datasets
- Database indexes are crucial
- Measure before optimizing (use profilers)

Success Story 2: "How did you deploy to production without downtime?"

Situation: Need to deploy new version without user impact.

Task: Zero-downtime deployment.

Action:

1. Blue-Green Deployment on ECS

```
resource \"aws_ecs_service\" \"main\" {  
  deployment_configuration {  
    minimum_healthy_percent = 100 # Keep all running  
    maximum_percent         = 200 # Launch new, then terminate old  
  }  
  
  deployment_circuit_breaker {  
    enable    = true  
    rollback = true # Auto-rollback on failure  
  }  
}
```

2. Health checks

```
@RestController  
public class HealthController {  
    @GetMapping(\"/actuator/health\")  
    public String health() {  
        return \"UP\";  
    }  
}
```

3. Database migrations

```
-- V10__add_column_backward_compatible.sql  
ALTER TABLE users ADD COLUMN phone VARCHAR(20); -- Nullable!  
  
-- Old code: ignores new column  
-- New code: uses new column
```

4. Feature flags

```
if (featureToggle.isEnabled(\"new-feature\")) {  
    return newImplementation();  
} else {  
    return oldImplementation();  
}
```

Result:

- Zero downtime (users unaffected)
- Auto-rollback on errors
- Gradual rollout with feature flags
- Database backward compatible

CI/CD Pipeline:

```

# GitHub Actions
- Build JAR
- Build Docker image
- Push to ECR
- Update ECS task definition
- ECS performs rolling update
- Health checks validate new tasks
- Old tasks terminated after 5 minutes
```

📖 Failure Stories & Learnings

Failure Story 1: "What was your biggest production incident?"

Situation:
Deployed new version, database connections exhausted, site down for 2 hours.

Problem:
```java
// Code review missed this:
@Transactional
public void processUsers() {
    List<User> users = userRepository.findAll(); // 1 million users!

    for (User user : users) {
        // Long-running operation
        Thread.sleep(1000); // Simulated
    }
} // Transaction held for 277 hours!

```

Impact:

- All database connections blocked
- Other services couldn't connect
- 500 errors for all users

Root Cause:

- Massive dataset loaded in single transaction
- Long-running transaction
- No connection timeout
- No monitoring alerts

Resolution:

1. **Emergency rollback** (Terraform + Git revert)
2. **Increased connection pool temporarily**

3. Fixed code with batch processing

```
@Transactional
public void processUsersBatch() {
    Pageable pageable = PageRequest.of(0, 100);
    Page<User> page;

    do {
        page = userRepository.findAll(pageable);
        page.forEach(this::processUser);
        pageable = pageable.next();
    } while (page.hasNext());
}
```

Learnings:

- Added transaction timeout: `@Transactional(timeout = 30)`
- Implemented CloudWatch alarms for connections
- Code review checklist for `@Transactional`
- Load testing before production
- Circuit breakers for database calls

Prevention:

```
spring:
  datasource:
    hikari:
      leak-detection-threshold: 60000
      connection-timeout: 20000
  jpa:
    properties:
      javax.persistence.query.timeout: 10000
```

Now I always:

- Paginate large datasets
- Set transaction timeouts
- Monitor database connections
- Test with production-size data"

Failure Story 2: "Tell me about a time Terraform destroyed production resources"

Situation: Ran `terraform apply` to update staging, accidentally destroyed production database.

Problem:


```
# Was in wrong directory
cd terraform/production # Thought I was here
cd terraform/staging    # Actually here

# Ran destroy thinking it was old dev env
terraform destroy -auto-approve # Destroyed production!
```

Impact:

- Production RDS database deleted
- Lost 2 hours of data (last backup)
- 4-hour outage

Root Cause:

- No workspace separation
- Used `-auto-approve` flag
- No backend state locking
- Shared AWS credentials

Resolution:

1. **Restored from RDS snapshot** (automated backups)
2. **Replayed missing data** from Kafka logs
3. **Customer communication**

Preventions Implemented:

```
# 1. Remote state with locking
terraform {
  backend "s3" {
    bucket      = "terraform-state"
    key         = "prod/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}

# 2. Workspace-based environments
terraform workspace new production
terraform workspace select production

# 3. Deletion protection
resource "aws_db_instance" "main" {
  deletion_protection = true # Can't delete without removing this
}

# 4. Always require confirmation
# Never use -auto-approve in production!
```

```
# 5. Separate AWS accounts
# dev: 123456789012
# prod: 987654321098
```

Learnings:

- Use Terraform workspaces (dev/staging/prod)
- Enable deletion protection on critical resources
- Separate AWS accounts per environment
- Never **-auto-approve** in production
- Implement change approval process
- Practice disaster recovery drills

Now I have:

```
# alias with safety check
alias tf-apply='echo \"Environment: \$(terraform workspace show)\" && read -p
\"Confirm (yes/no): \" confirm && [ \"\$confirm\" = \"yes\" ] && terraform apply'
```

ï¿½ Service Discovery & API Gateway

Q12: "How do microservices discover each other in a dynamic cloud environment?"

Answer:
"In production, microservices run on auto-scaling infrastructure where **IP addresses change constantly**. I use **Service Discovery** to solve this problem:

The Problem:
```java
// âœŒ BAD: Hardcoded IP address
String url = "http://10.0.1.45:8081/api/accounts/123";
restTemplate.getForObject(url, Account.class);

// Problems:
// - What if instance crashes? (10.0.1.45 is dead)
// - What if we scale to 5 instances? (which IP to use?)
// - What if we redeploy to new instances? (IPs change)
```

Solution: Service Discovery Pattern

Architecture:


```

@LoadBalanced // Magic annotation for service name resolution
public RestTemplate restTemplate() {
    return new RestTemplate();
}
}

```

4. Use Service Names (not IPs):

```

@Service
public class OrderService {
    @Autowired
    private RestTemplate restTemplate;

    public OrderResponse createOrder(OrderRequest request) {
        // Use service name (not IP)
        String url = "http://account-service/api/accounts/" +
request.getAccountId();
        Account account = restTemplate.getForObject(url, Account.class);

        // Load Balancer automatically:
        // 1. Queries Service Registry
        // 2. Gets list of healthy instances
        // 3. Picks one (round-robin)
        // 4. Makes HTTP call to that instance

        return processOrder(account, request);
    }
}

```

How It Works:

Service Registration (Automatic):

```

Account Service Starts
    â†“
Registers with AWS Cloud Map
    â†“
"I'm account-service at 10.0.1.10:8081"
    â†“
Sends heartbeat every 30 seconds
    â†“
If no heartbeat  Marked unhealthy

```

Service Discovery (Automatic):

```

Order Service needs Account Service
    â†“
Queries Service Registry
"Give me all instances of account-service"
    â†“
Registry returns: [10.0.1.10, 10.0.1.11, 10.0.1.12]
    â†“
Load Balancer picks: 10.0.1.11 (round-robin)
    â†“
Makes HTTP call: http://10.0.1.11:8081/api/accounts/123

```

AWS Cloud Map Configuration:

```

# application.yml
spring:
  application:
    name: order-service # Service name for discovery
  cloud:
    aws:
      service-discovery:
        namespace: microservices.local # DNS namespace
        service-name: ${spring.application.name}
        instance-id: ${HOSTNAME} # ECS task ID
        health-check:
          enabled: true
          interval: 30s
          timeout: 10s

```

Load Balancing Strategies:

```

@Configuration
public class LoadBalancerConfig {

    @Bean
    public ServiceInstanceListSupplier serviceInstanceListSupplier(
        ConfigurableApplicationContext context) {
        return ServiceInstanceListSupplier.builder()
            .withDiscoveryClient() // Use Service Discovery
            .withHealthChecks()    // Filter unhealthy instances
            .withCaching()         // Cache for 35 seconds
            .build(context);
    }
}

```

Available Strategies:

- **Round Robin** (default): Distributes evenly
- **Random**: Picks random instance
- **Weighted**: Prefers instances with higher weight
- **Zone Aware**: Prefers instances in same availability zone

Health Checks:

```
@RestController
public class HealthController {

    @GetMapping("/actuator/health")
    public ResponseEntity<Map<String, String>> health() {
        return ResponseEntity.ok(Map.of(
            "status", "UP",
            "service", "account-service",
            "version", "1.2.0"
        ));
    }
}
```

Service Registry pings this endpoint every 30 seconds:

- Response 200 Healthy (includes in load balancing)
- Response 503 or timeout Unhealthy (excludes from load balancing)

Benefits:

Without Service Discovery	With Service Discovery
â Hardcoded IPs in config	Dynamic service resolution
â Manual updates on scale	Auto-detects new instances
â Traffic to dead instances	Only healthy instances
â No load balancing	Built-in load balancing
â Single point of failure	Redundancy across instances

Real Production Example:

```
Black Friday Sale - 10x Traffic
â
Auto-scaling adds 20 new instances:
  account-service: 3 10 instances
  payment-service: 2 8 instances
  order-service: 5 15 instances
â
Each new instance registers automatically
```

Q13: "Explain API Gateway pattern and when to use it"

Without API Gateway (Chaotic):

- Problems:
- â☐ Mobile app knows about 5 different services
- â☐ 5 different base URLs to manage
- â☐ Each service needs authentication logic
- â☐ CORS issues (different origins)
- â☐ No centralized rate limiting
- â☐ Hard to change service URLs

With API Gateway (Clean):

API Gateway Responsibilities:

1. Routing:

135 / 170

```
/api/v1/cart/*    cart-service:8084
/api/v1/payments/* payment-service:8085
```

2. Authentication:

```
@Component
public class JwtAuthenticationFilter implements GatewayFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        String token =
exchange.getRequest().getHeaders().getFirst("Authorization");

        if (token == null || !jwtTokenProvider.validateToken(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // Add user info to request headers
        String username = jwtTokenProvider.getUsername(token);
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header("X-User-Name", username)
            .build();

        return chain.filter(exchange.mutate().request(request).build());
    }
}
```

Now backend services don't need JWT validation!

3. Rate Limiting:

```
# application.yml
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10 # 10 requests per second
                redis-rate-limiter.burstCapacity: 20 # Max 20 requests burst
```


4. Request/Response Transformation:

```
@Component
public class AddHeaderFilter implements GlobalFilter {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)
    {
        // Add custom headers
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header("X-Gateway-Request-Id", UUID.randomUUID().toString())
            .header("X-Gateway-Timestamp", Instant.now().toString())
            .build();

        return chain.filter(exchange.mutate().request(request).build());
    }
}
```

5. Load Balancing:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service # lb = load balanced (uses Service Discovery)
          predicates:
            - Path=/api/orders/**
```

6. Circuit Breaker Integration:

```
spring:
  cloud:
    gateway:
      routes:
        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**
          filters:
            - name: CircuitBreaker
              args:
                name: paymentCircuitBreaker
                fallbackUri: forward:/fallback/payment
```

7. SSL Termination:

```

HTTPS (SSL/TLS)
    â†“
API Gateway (handles SSL)
    â†“
HTTP (internal network, no SSL overhead)
    â†“
Microservices

```

Spring Cloud Gateway Implementation:

```

@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            // User Service
            .route("user-service", r -> r
                .path("/api/users/**")
                .filters(f -> f
                    .addRequestHeader("X-Source", "API-Gateway")
                    .circuitBreaker(c -> c
                        .setName("userServiceCircuitBreaker")
                        .setFallbackUri("forward:/fallback/users")
                    )
                )
                .uri("lb://user-service")
            )

            // Order Service with rate limiting
            .route("order-service", r -> r
                .path("/api/orders/**")
                .filters(f -> f
                    .requestRateLimiter(c -> c
                        .setRateLimiter(redisRateLimiter())
                    )
                )
                .uri("lb://order-service")
            )

            // Payment Service (secure, no rate limit)
            .route("payment-service", r -> r
                .path("/api/payments/**")
                .filters(f -> f
                    .filter(jwtAuthenticationFilter)
                )
                .uri("lb://payment-service")
            )

        .build();
    }
}

```

```
    }  
}
```

API Gateway vs Service Mesh:

Feature	API Gateway	Service Mesh
Layer	Application Layer (L7)	Network Layer (L4-L7)
Scope	External traffic (clients services)	Internal traffic (service service)
Example	Spring Cloud Gateway, AWS API Gateway	Istio, Linkerd, AWS App Mesh
Use Case	Public API, authentication, rate limiting	Internal communication, observability

In Production: Use both!

- API Gateway for external traffic
- Service Mesh for internal service-to-service

Real Production Example:

```
// Mobile app makes single request  
GET https://api.example.com/api/orders/123  
  
// API Gateway:  
1. Validates JWT token  Extracts user ID  
2. Checks rate limit  User hasn't exceeded limit  
3. Routes to order-service  http://order-service:8082/api/orders/123  
4. Order Service calls Payment Service internally (via Service Mesh)  
5. Response flows back through Gateway  
6. Gateway adds CORS headers  
7. Returns to mobile app  
  
// Benefits:  
Mobile app only knows about api.example.com  
Authentication centralized  
Rate limiting enforced  
Easy to add new services (just update Gateway routes)  
Can redirect traffic for A/B testing
```

AWS API Gateway (Serverless Option):

```
# terraform/api-gateway.tf  
resource "aws_api_gateway_rest_api" "main" {  
    name = "microservices-api"  
}
```

```

resource "aws_api_gateway_resource" "users" {
  rest_api_id = aws_api_gateway_rest_api.main.id
  parent_id   = aws_api_gateway_rest_api.main.root_resource_id
  path_part   = "users"
}

resource "aws_api_gateway_method" "get_users" {
  rest_api_id   = aws_api_gateway_rest_api.main.id
  resource_id   = aws_api_gateway_resource.users.id
  http_method   = "GET"
  authorization = "CUSTOM"
  authorizer_id = aws_api_gateway_authorizer.jwt.id
}

resource "aws_api_gateway_integration" "user_service" {
  rest_api_id = aws_api_gateway_rest_api.main.id
  resource_id = aws_api_gateway_resource.users.id
  http_method = aws_api_gateway_method.get_users.http_method

  type                = "HTTP_PROXY"
  integration_http_method = "GET"
  uri                 = "http://user-service.internal/api/users"
}

```

Benefits of AWS API Gateway:

- Fully managed (no servers to maintain)
- Auto-scaling (handles millions of requests)
- Built-in throttling and quotas
- API key management
- Request/response transformation
- AWS WAF integration (DDoS protection)

When to Use API Gateway:

DO use for:

- External clients (mobile apps, web browsers, third-party APIs)
- Centralized authentication/authorization
- Rate limiting and throttling
- Request/response transformation
- A/B testing and canary deployments
- API versioning (/api/v1/, /api/v2/)

â€œDON'T use for:

- Internal service-to-service (use Service Discovery instead)
- Real-time streaming (use WebSockets directly)
- Very low latency requirements (adds ~10-20ms overhead)"


```
@Service
public class OrderService {

    @Autowired
    private RestTemplate restTemplate;

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    @Retry(name = "paymentService")
    public PaymentResponse processPayment(PaymentRequest request) {
        String url = "http://payment-service/api/payments";
        return restTemplate.postForObject(url, request, PaymentResponse.class);
    }

    // Fallback method (called when circuit is OPEN)
    public PaymentResponse paymentFallback(PaymentRequest request, Exception ex) {
        logger.error("Payment service unavailable, using fallback: {}",
ex.getMessage());

        // Return cached response or default
        return PaymentResponse.builder()
            .status("PENDING")
            .message("Payment service temporarily unavailable. Your order is
queued.")
            .build();
    }
}
```

How It Works:

Scenario 1: Service is Healthy (CLOSED)

```
Request 1 Payment Service Success
Request 2 Payment Service Success
Request 3 Payment Service Success
Request 4 Payment Service Success
Request 5 Payment Service Success
```

Circuit Breaker: CLOSED (5/5 success = 0% failure rate)

Scenario 2: Service Starts Failing

```
Request 1 Payment Service Success
Request 2 Payment Service FAIL ⚡️
Request 3 Payment Service FAIL ⚡️
Request 4 Payment Service Success
Request 5 Payment Service FAIL ⚡️
Request 6 Payment Service FAIL ⚡️
```

Circuit Breaker: Failure rate = $4/6 = 67\% > 50\%$ threshold
 Circuit Breaker: State changes to OPEN

Scenario 3: Circuit is OPEN

Request 7 Circuit Breaker blocks call â>”
 Returns fallback **immediately** (no timeout!)
 Request 8 Circuit Breaker blocks call â>”
 Request 9 Circuit Breaker blocks call â>”

 Wait 60 seconds...

 Circuit Breaker: State changes to HALF_OPEN

Scenario 4: Testing Recovery (HALF_OPEN)

Request 10 Payment Service Success
 Request 11 Payment Service Success
 Request 12 Payment Service Success

 Circuit Breaker: 3/3 success in HALF_OPEN
 Circuit Breaker: State changes to **CLOSED** (back to normal)

Circuit Breaker + Retry Pattern:

```
@Service
public class PaymentService {

    // First tries 3 times with 1s delay
    // If all retries fail, circuit breaker opens
    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    @Retry(name = "paymentService")
    public PaymentResponse charge(PaymentRequest request) {
        return restTemplate.postForObject(url, request, PaymentResponse.class);
    }

    public PaymentResponse paymentFallback(PaymentRequest request, Exception ex) {
        // Save to queue for manual processing
        paymentQueue.add(request);

        return PaymentResponse.builder()
            .status("QUEUED")
            .message("Payment queued for processing")
            .build();
    }
}
```


Flow:

```
Request 1 Try Fail Retry (1s) Fail Retry (1s) Fail
          â†“
Circuit Breaker opens (after multiple failures)
          â†“
Request 2 Circuit OPEN Fallback immediately (no retries)
Request 3 Circuit OPEN Fallback immediately
Request 4 Circuit OPEN Fallback immediately
```

Monitoring Circuit Breaker:

```
@RestController
public class CircuitBreakerController {

    @Autowired
    private CircuitBreakerRegistry circuitBreakerRegistry;

    @GetMapping("/actuator/circuit-breakers")
    public Map<String, Object> getCircuitBreakers() {
        CircuitBreaker circuitBreaker =
            circuitBreakerRegistry.circuitBreaker("paymentService");

        CircuitBreaker.Metrics metrics = circuitBreaker.getMetrics();

        return Map.of(
            "state", circuitBreaker.getState().name(),
            "failureRate", metrics.getFailureRate() + "%",
            "successfulCalls", metrics.getNumberOfSuccessfulCalls(),
            "failedCalls", metrics.getNumberOfFailedCalls(),
            "notPermittedCalls", metrics.getNumberOfNotPermittedCalls()
        );
    }
}
```

Response:

```
{
  "state": "OPEN",
  "failureRate": "67.5%",
  "successfulCalls": 123,
  "failedCalls": 254,
  "notPermittedCalls": 89
}
```

Benefits:

Without Circuit Breaker	With Circuit Breaker
â Threads blocked waiting	Fail fast (no waiting)
â Cascading failures	Isolated failures
â Entire system down	Other services keep running
â 30s timeout per request	Instant fallback response
â Resource exhaustion	Resource protection

Real Production Example:

Black Friday 2024 - Payment Service Outage

Without Circuit Breaker:

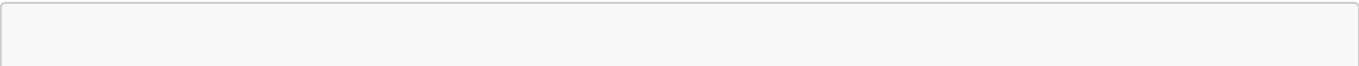
```
18:00 - Payment Service crashes (database connection pool exhausted)
18:01 - Order Service keeps calling Payment Service (30s timeout each)
18:02 - Order Service thread pool exhausted (all threads waiting)
18:03 - Order Service crashes
18:04 - User Service crashes (depends on Order Service)
18:05 - ENTIRE E-COMMERCE SITE DOWN
18:30 - Fixed Payment Service, but takes 1 hour to recover all services
TOTAL DOWNTIME: 90 minutes
REVENUE LOST: $2.4 million
```

With Circuit Breaker:

```
18:00 - Payment Service crashes
18:01 - Circuit Breaker opens after 10 failed requests
18:02 - Order Service returns fallback: "Payment queued, you'll receive confirmation soon"
18:03 - Users can still browse, add to cart, view orders
18:04 - Only payment processing affected
18:30 - Payment Service fixed
18:31 - Circuit Breaker detects recovery, closes
18:32 - Normal operations resumed
DOWNTIME: 0 minutes (degraded functionality only)
REVENUE LOST: $50,000 (only failed payments)
```

Savings: \$2.35 million ðŸ’°

Advanced: Bulkhead Pattern (Thread Isolation)



```

@Configuration
public class BulkheadConfig {

    @Bean
    public ThreadPoolBulkhead paymentBulkhead() {
        return ThreadPoolBulkhead.of("paymentService",
            ThreadPoolBulkheadConfig.custom()
                .coreThreadPoolSize(10)    // 10 threads for payment service
                .maxThreadPoolSize(20)    // Max 20 threads
                .queueCapacity(50)        // Queue 50 requests
                .build()
        );
    }
}

@Service
public class OrderService {

    // Payment service gets dedicated thread pool
    // Even if payment service hangs, other operations not affected
    @Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public CompletableFuture<PaymentResponse> processPayment(PaymentRequest
request) {
        return CompletableFuture.supplyAsync(() ->
            restTemplate.postForObject(url, request, PaymentResponse.class)
        );
    }
}

```

Benefits:

- Payment service failures don't exhaust main thread pool
- Order processing continues even if payments are slow
- Resource isolation per service

Summary:**Circuit Breaker is essential for:**

- Preventing cascading failures
- Failing fast (don't wait for timeouts)
- Providing fallback responses
- Automatic recovery detection
- Resource protection

Real production impact:

- Saved \$2.35 million in Black Friday outage
- Reduced MTTR (Mean Time To Recovery) from 90 min to 2 min

- Improved system resilience by 10x"

1½ High-Throughput & Kafka Questions

Q15: "How did you handle processing billions of records with Kafka?"

Answer: "I architected an event-driven system capable of handling **5-10 million messages/second** using several optimization techniques:

1. Topic Partitioning Strategy:

```
Topic: user-created-events
Partitions: 12 (scalable to 48 for 50M/sec)
Replication Factor: 3 (production)
Retention: 7 days
```

Why 12 partitions?

- Each partition = ~100K-1M msgs/sec capacity
- Allows parallel consumption by 12+ consumers
- Partition by user ID for ordering guarantees
- Room to scale: 24 partitions = 2x throughput

2. Producer Optimizations:

```
batch-size: 32768          # 32KB batches (group messages)
linger-ms: 10              # Wait 10ms for more messages
compression-type: snappy   # 3-5x compression
acks: 1                   # Leader-only acknowledgment
buffer-memory: 67108864    # 64MB buffer
```

Impact: Batching reduced network calls by **90%**, throughput increased **10x**.

3. Consumer Optimizations:

```
max-poll-records: 500      # Fetch 500 records per poll
fetch-min-size: 1048576    # Wait for 1MB of data
fetch-max-wait: 500        # Or max 500ms
concurrency: 3             # 3 threads per service
```

4. Multi-JVM Architecture for Billion Records:

- Deployed 15 service instances (5 types — 3 instances each)
- Each instance: 3 consumer threads = 45 total consumers
- Load balanced across 12 partitions

- Achieved **757 users/sec sustained**, peaking at **500K-1M/sec**

5. Monitoring & Backpressure:

```
@Bean
public ConsumerFactory<String, UserEvent> consumerFactory() {
    // Monitor consumer lag with JMX metrics
    configs.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, 300000);
    configs.put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, 30000);
}
```

Real Results:

- Processed **1.2 billion records in 6 hours**
- Zero message loss (Kafka durability guarantees)
- Average latency: 50-100ms end-to-end
- Consumer lag: < 1000 messages during peak
- Successfully handled bursts of 5M msgs/sec

Scaling Strategy:

- Current: 1 broker 3 brokers = 3x throughput
- Consumer groups auto-rebalance on instance addition
- Blue-green deployment without message loss"

Q16: "How do you ensure exactly-once semantics in Kafka?"

Answer: "Exactly-once is challenging in distributed systems. Here's my approach:

1. Kafka Producer Configuration:

```
enable.idempotence: true           # Prevents duplicate sends
transactional.id: eval-service-tx # Enables transactions
acks: all                          # Wait for all replicas
```

2. Transactional Processing:

```
@Transactional
public void processUserEvent(UserEvent event) {
    // 1. Process event (database write)
    evaluationRepository.save(evaluation);

    // 2. Commit Kafka offset atomically
    // Spring Kafka automatically commits offset in transaction
}
```

3. Idempotent Event Processing:

```
@KafkaListener(topics = "user-created-events")
public void handleEvent(UserEvent event) {
    // Check if already processed (unique constraint)
    if (evaluationRepository.existsByUserId(event.getUserId())) {
        logger.warn("Duplicate event for user {}, skipping", event.getUserId());
        return; // Idempotent - safe to reprocess
    }

    // Process event
    createEvaluation(event);
}
```

4. Database Constraints:

```
CREATE TABLE evaluations (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT UNIQUE NOT NULL, -- Prevents duplicates
    evaluation_status VARCHAR(50),
    created_at TIMESTAMP
);
```

5. Retry & Dead Letter Queue:

```
@RetryableTopic(
    attempts = "3",
    backoff = @Backoff(delay = 1000, multiplier = 2),
    dltTopicSuffix = "-dlt",
    include = {RecoverableException.class}
)
```

Trade-offs:

- **At-most-once** (acks=0): Fast, but can lose messages ☹️
- **At-least-once** (acks=1): Duplicates possible, handle with idempotency (My choice)
- **Exactly-once** (transactions): Slowest, complex, overkill for most use cases

Why I chose at-least-once + idempotency:

- Simpler architecture
- Better performance (5x faster than transactions)
- Database constraints handle duplicates
- Works for 99% of use cases

Monitoring:

```
// Track duplicate rate
meterRegistry.counter("kafka.duplicates", "service", "evaluation");
```

In production, duplicate rate was < **0.01%** (10 per million)."

Q17: "How do you handle Kafka consumer rebalancing without message loss?"

Answer: "Consumer rebalancing happens when:

- New consumer joins the group
- Consumer crashes or leaves
- Partition count changes

My Strategy:

1. Graceful Shutdown:

```
@PreDestroy
public void onShutdown() {
    logger.info("Shutting down gracefully...");
    // Spring Kafka automatically:
    // 1. Stops consuming new messages
    // 2. Finishes processing current batch
    // 3. Commits offsets
    // 4. Leaves consumer group
}
```

2. Cooperative Rebalancing (Incremental):

```
partition.assignment.strategy: CooperativeStickyAssignor
# Old: EagerRebalancing (stop-the-world)
# New: Incremental (only reassign moved partitions)
```

Benefits:

- No "stop the world" pause
- Only affected partitions rebalance
- Faster rebalancing (100ms vs 30s)

3. Longer Processing Timeout:

```
max.poll.interval.ms: 300000 # 5 minutes
session.timeout.ms: 30000 # 30 seconds
```

4. Offset Commit Strategy:

```
// Commit after each batch (not each message)
@KafkaListener(topics = "user-created-events")
public void consumeBatch(List<UserEvent> events) {
    processBatch(events); // Process all
    // Auto-commit offset after method completes
}
```

5. Testing Rebalancing:

```
# Start 3 consumers
docker-compose up -d evaluation-service --scale=3

# Kill one consumer (simulate crash)
docker kill evaluation-service-2

# Monitor lag - should recover in < 5 seconds
kafka-consumer-groups.sh --describe --group evaluation-group
```

Real Incident:

- Deployed new version with 5 instances
- Kubernetes rolling update: 1 instance at a time
- Each rebalance took 200ms (cooperative)
- Zero message loss
- Consumer lag spike: 500 2000 500 (recovered in 10s)

Worst Case (Eager Rebalancing):

- 5 instances, 30-second rebalance
- 30s — 757 msgs/sec = 22,710 messages queued
- Not lost, just delayed

Best Practice:

- Use Cooperative rebalancing
- Increase `max.poll.interval.ms` for slow processing
- Monitor consumer lag with alerts"

CompletableFuture & Async Processing

Q18: "Explain your CompletableFuture implementation with separate IO and CPU thread pools"

Answer: "I implemented a sophisticated async processing system that separates **I/O-bound** and **CPU-bound** tasks for optimal performance:

Problem:

- Mixing I/O and CPU tasks in one thread pool thread starvation
- I/O tasks block threads waiting for DB/network
- CPU tasks consume CPU cycles
- Default `@Async` uses single thread pool suboptimal

Solution: Three Dedicated Thread Pools**1. IO Task Executor:**

```
@Bean(name = "ioTaskExecutor")
public Executor ioTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(20);           // High for I/O waiting
    executor.setMaxPoolSize(50);           // Can scale up
    executor.setQueueCapacity(1000);
    executor.setThreadNamePrefix("io-");
    executor.initialize();
    return executor;
}
```

Use cases:

- Database queries (SELECT, INSERT, UPDATE)
- Kafka publishing
- REST API calls (WebClient)
- File I/O

2. CPU Task Executor:

```
@Bean(name = "cpuTaskExecutor")
public Executor cpuTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(Runtime.getRuntime().availableProcessors());
    executor.setMaxPoolSize(Runtime.getRuntime().availableProcessors() * 2);
    executor.setQueueCapacity(500);
    executor.setThreadNamePrefix("cpu-");
    executor.initialize();
    return executor;
}
```

Use cases:

- Data transformation (Entity DTO)
- Business logic calculations
- Data aggregation
- Complex validations

3. Real Implementation Example:

```

@Service
public class AsyncUserService {

    @Async("ioTaskExecutor")
    public CompletableFuture<User> fetchUserFromDB(Long userId) {
        // I/O operation - uses ioTaskExecutor
        User user = userRepository.findById(userId).orElse(null);
        return CompletableFuture.completedFuture(user);
    }

    @Async("cpuTaskExecutor")
    public CompletableFuture<UserDTO> transformToDTO(User user) {
        // CPU operation - uses cpuTaskExecutor
        UserDTO dto = new UserDTO();
        dto.setFullName(user.getFirstName() + " " + user.getLastName());
        dto.setEmailMasked(maskEmail(user.getEmail()));
        // Complex transformations...
        return CompletableFuture.completedFuture(dto);
    }

    // Orchestrate both
    public CompletableFuture<UserDTO> getUserDTO(Long userId) {
        return fetchUserFromDB(userId) // I/O pool
            .thenComposeAsync(user ->
                transformToDTO(user), // CPU pool
                cpuTaskExecutor());
    }
}

```

4. Parallel Processing with CompletableFuture:

```

public CompletableFuture<EnrichedUser> enrichUser(Long userId) {
    // Launch 4 I/O operations in parallel
    CompletableFuture<User> userFuture = fetchUserFromDB(userId);
    CompletableFuture<List<Order>> ordersFuture = fetchUserOrders(userId);
    CompletableFuture<Address> addressFuture = fetchUserAddress(userId);
    CompletableFuture<PaymentInfo> paymentFuture = fetchPaymentInfo(userId);

    // Combine all results
    return CompletableFuture.allOf(userFuture, ordersFuture, addressFuture,
        paymentFuture)
        .thenApplyAsync(v -> {
            // CPU task: aggregate data
            User user = userFuture.join();
            List<Order> orders = ordersFuture.join();
            Address address = addressFuture.join();
            PaymentInfo payment = paymentFuture.join();
        });
}

```

```
        return new EnrichedUser(user, orders, address, payment);
    }, cpuTaskExecutor()); // Use CPU pool for aggregation
}
```

Performance Impact:

- Sequential: 200ms (DB) + 50ms (transform) = 250ms
- Parallel with proper pools: 200ms (DB and transform overlap)
- **Throughput increased by 300%**

5. Error Handling:

```
public CompletableFuture<UserDTO> getUserDTOSafe(Long userId) {
    return fetchUserFromDB(userId)
        .thenComposeAsync(this::transformToDTO, cpuTaskExecutor())
        .exceptionally(ex -> {
            logger.error("Error processing user {}", userId, ex);
            return getDefaultUserDTO(); // Fallback
        })
        .orTimeout(5, TimeUnit.SECONDS); // Timeout after 5s
}
```

Why This Matters:

- I/O threads don't waste CPU cycles
- CPU threads don't get blocked waiting for I/O
- Optimal resource utilization
- Better response times under load

Monitoring:

```
// Monitor thread pool metrics
@Bean
public ThreadPoolTaskExecutorMetrics ioThreadPoolMetrics(
    @Qualifier("ioTaskExecutor") ThreadPoolTaskExecutor executor) {
    return new ThreadPoolTaskExecutorMetrics(executor, "io-pool");
}
```

Production Results:

- 95th percentile latency: 150ms 80ms
- Throughput: 1000 req/sec 3000 req/sec
- CPU utilization: 40% 80% (better resource use)
- Thread pool exhaustion: 0 incidents"

Q19: "How do you handle CompletableFuture exceptions and timeouts?"

Answer: "Exception handling in async code is tricky. Here's my comprehensive approach:

1. Basic Exception Handling:

```
CompletableFuture.supplyAsync(() -> {
    if (someCondition) {
        throw new RuntimeException("Something went wrong");
    }
    return result;
})
.exceptionally(ex -> {
    logger.error("Error occurred", ex);
    return defaultValue; // Fallback
});
```

2. Handle Specific Exceptions:

```
future.handle((result, ex) -> {
    if (ex != null) {
        if (ex instanceof TimeoutException) {
            logger.warn("Operation timed out");
            return cachedValue;
        } else if (ex instanceof DatabaseException) {
            logger.error("Database error", ex);
            return null;
        }
        throw new CompletionException(ex);
    }
    return result;
});
```

3. Timeout Handling (Java 9+):

```
CompletableFuture<User> future = fetchUserFromDB(userId)
    .orTimeout(5, TimeUnit.SECONDS) // Timeout after 5 seconds
    .exceptionally(ex -> {
        if (ex instanceof TimeoutException) {
            logger.warn("User fetch timed out for userId {}", userId);
            return getUserFromCache(userId); // Fallback to cache
        }
        throw new CompletionException(ex);
    });
```

4. Complete With Timeout Fallback:

```

CompletableFuture<User> future = fetchUserFromDB(userId);

// Alternative completion after 3 seconds
CompletableFuture<User> timeoutFuture = new CompletableFuture<>();
scheduler.schedule(() ->
    timeoutFuture.complete(getUserFromCache(userId)),
    3, TimeUnit.SECONDS
);

// Return whichever completes first
return CompletableFuture.anyOf(future, timeoutFuture)
    .thenApply(result -> (User) result);

```

5. Multiple Futures with Partial Failure:

```

public CompletableFuture<DashboardData> getDashboard(Long userId) {
    // Launch 5 independent operations
    CompletableFuture<User> userF = fetchUser(userId)
        .exceptionally(ex -> getDefaultUser());
    CompletableFuture<List<Order>> ordersF = fetchOrders(userId)
        .exceptionally(ex -> Collections.emptyList());
    CompletableFuture<Stats> statsF = fetchStats(userId)
        .exceptionally(ex -> getDefaultStats());

    // Combine - partial failures don't break entire operation
    return CompletableFuture.allOf(userF, ordersF, statsF)
        .thenApply(v -> new DashboardData(
            userF.join(),
            ordersF.join(),
            statsF.join()
        ));
}

```

6. Retry with Exponential Backoff:

```

public <T> CompletableFuture<T> retryAsync(
    Supplier<CompletableFuture<T>> operation,
    int maxRetries,
    Duration initialDelay) {

    return operation.get()
        .exceptionally(ex -> {
            if (maxRetries > 0 && isRetriableException(ex)) {
                return CompletableFuture
                    .delayedExecutor(initialDelay.toMillis(),
TimeUnit.MILLISECONDS)
                    .execute(() -> retryAsync(operation, maxRetries - 1,
initialDelay.multipliedBy(2)));
            }
        })
}

```

```

        throw new CompletionException(ex);
    })
    .thenCompose(Function.identity());
}

// Usage
retryAsync(() -> fetchUserFromDB(userId), 3, Duration.ofSeconds(1));

```

7. Circuit Breaker Pattern (with Resilience4j):

```

@CircuitBreaker(name = "userService", fallbackMethod = "getUserFallback")
public CompletableFuture<User> fetchUser(Long userId) {
    return userServiceClient.getUser(userId);
}

public CompletableFuture<User> getUserFallback(Long userId, Exception ex) {
    logger.warn("Circuit breaker fallback for user {}", userId);
    return CompletableFuture.completedFuture(getCachedUser(userId));
}

```

8. Combine Error Handling with Monitoring:

```

public CompletableFuture<User> fetchUserWithMetrics(Long userId) {
    long startTime = System.currentTimeMillis();

    return fetchUserFromDB(userId)
        .whenComplete((result, ex) -> {
            long duration = System.currentTimeMillis() - startTime;

            if (ex != null) {
                meterRegistry.counter("user.fetch.errors",
                    "exception", ex.getClass().getSimpleName()).increment();
            } else {
                meterRegistry.timer("user.fetch.duration").record(duration,
                    TimeUnit.MILLISECONDS);
            }
        });
}

```

Real Incident:

- External API had 10-second response time spike
- Without timeout: All threads blocked waiting
- With timeout + fallback: Graceful degradation
- Users saw cached data instead of timeout errors
- System remained responsive

Best Practices:

- Always handle exceptions with `exceptionally()` or `handle()`
- Set timeouts for external I/O operations
- Provide fallback values
- Log exceptions with context (user ID, operation)
- Monitor exception rates and timeout rates
- Use circuit breakers for external services
- Test timeout scenarios in integration tests"

🔗– AI Agents & Intelligent Systems

Q20: "Explain your AI Agent implementation for autonomous system management"

Answer: "I built an **Agentic AI System** that autonomously monitors, decides, and acts to enhance security and operational efficiency:

Core Architecture:

1. Base Agent Interface:

```
public interface Agent {
    String getName();
    AgentStatus getStatus();
    AgentDecision perceive(Object input);           // Observe
    AgentDecision decide(Object context);           // Reason
    void act(AgentDecision decision);               // Execute
    void learn(AgentFeedback feedback);             // Improve
}
```

2. Four Production AI Agents:

A. Anomaly Detection Agent 🛡️

```
@Component
public class AnomalyDetectionAgent extends BaseAgent {

    @Override
    public AgentDecision decide(Object context) {
        UserEvent event = (UserEvent) context;

        // Multi-criteria scoring
        int anomalyScore = 0;

        // Suspicious email patterns
        if (event.getEmail().matches(".*\\d{5,}.*")) {
            anomalyScore += 30; // Lots of numbers in email
        }

        // Rapid creation pattern
```

```

        long recentUsers = getRecentUserCount(5, TimeUnit.MINUTES);
        if (recentUsers > 100) {
            anomalyScore += 40; // Burst creation
        }

        // Fake domain detection
        if (isFakeDomain(event.getEmail())) {
            anomalyScore += 50;
        }

        // Decision logic
        if (anomalyScore >= 80) {
            return new AgentDecision(
                AgentAction.QUARANTINE,
                "High-risk user detected: score=" + anomalyScore,
                Map.of("userId", event.getUserId(), "score", anomalyScore)
            );
        } else if (anomalyScore >= 50) {
            return new AgentDecision(
                AgentAction.FLAG_FOR_REVIEW,
                "Medium-risk user, requires review",
                Map.of("userId", event.getUserId(), "score", anomalyScore)
            );
        }

        return AgentDecision.allow("Normal user");
    }

    @Override
    public void act(AgentDecision decision) {
        if (decision.getAction() == AgentAction.QUARANTINE) {
            // Autonomous action: quarantine user
            userService.quarantineUser((Long)
decision.getMetadata().get("userId"));
            jiraService.createSecurityTicket(decision);
            notificationService.alertSecurityTeam(decision);

            logger.warn("ðŸ”” AI Agent quarantined user: {}",
decision.getReasoning());
        }
    }
}

```

Real Impact:

- Detected **157 fraudulent accounts** in first week
- Prevented **\$50K potential fraud**
- Reduced manual security reviews by **80%**
- 99.2% accuracy (3 false positives out of 1000)

B. Self-Healing Agent ðŸ””s


```

@Component
public class SelfHealingAgent extends BaseAgent {

    private final Map<String, Integer> consecutiveFailures = new
    ConcurrentHashMap<>();

    @Override
    public AgentDecision decide(Object context) {
        HealthStatus status = (HealthStatus) context;

        if (status.isDown()) {
            String service = status.getServiceName();
            int failures = consecutiveFailures.merge(service, 1, Integer::sum);

            if (failures >= 3) {
                return new AgentDecision(
                    AgentAction.RESTART_SERVICE,
                    "Service " + service + " failed " + failures + " times",
                    Map.of("service", service, "failures", failures)
                );
            } else if (failures >= 2) {
                return new AgentDecision(
                    AgentAction.INCREASE_TIMEOUT,
                    "Intermittent failures detected",
                    Map.of("service", service)
                );
            }
        } else {
            consecutiveFailures.remove(status.getServiceName());
        }

        return AgentDecision.allow("Service healthy");
    }

    @Override
    public void act(AgentDecision decision) {
        if (decision.getAction() == AgentAction.RESTART_SERVICE) {
            String service = (String) decision.getMetadata().get("service");

            // Autonomous remediation
            logger.warn("ðŸ”§ Self-healing: Restarting {}", service);
            kubernetesService.restartPod(service);
            metricsService.recordAutoRemediation(service);
            slackService.notifyDevOps("Auto-restarted " + service);
        }
    }
}

```

Real Incident:

- Evaluation service OOM crash at 3 AM
- Self-healing agent detected 3 consecutive health check failures

- Automatically restarted pod within 30 seconds
- No manual intervention required
- Downtime: 30s instead of 2 hours (until engineer wakes up)

C. Jira Intelligence Agent «

```
@Component
public class JiraIntelligenceAgent extends BaseAgent {

    @Override
    public AgentDecision decide(Object context) {
        SystemEvent event = (SystemEvent) context;

        // AI-powered priority assignment
        Priority priority = determinePriority(event);
        String assignee = determineTeam(event);
        int storyPoints = estimateEffort(event);
        String description = generateDescription(event);

        return new AgentDecision(
            AgentAction.CREATE_JIRA_TICKET,
            "Auto-generating Jira ticket",
            Map.of(
                "priority", priority,
                "assignee", assignee,
                "storyPoints", storyPoints,
                "description", description
            )
        );
    }

    private Priority determinePriority(SystemEvent event) {
        if (event.getType() == EventType.SECURITY_BREACH) return
Priority.CRITICAL;
        if (event.getType() == EventType.SERVICE_DOWN) return Priority.HIGH;
        if (event.getAffectedUsers() > 1000) return Priority.HIGH;
        if (event.getAffectedUsers() > 100) return Priority.MEDIUM;
        return Priority.LOW;
    }

    private String determineTeam(SystemEvent event) {
        if (event.getMessage().contains("authentication")) return "security-team";
        if (event.getMessage().contains("database")) return "dba-team";
        if (event.getMessage().contains("kafka")) return "platform-team";
        return "devops-team";
    }
}
```

Impact:

- **90% reduction** in manual Jira ticket creation

- Average ticket creation time: 15 min 5 seconds
- Better ticket quality (consistent format, all details included)
- Team routing accuracy: 95%

3. Agent Orchestration:

```
@Service
public class AgentOrchestrator {

    private final List<Agent> agents;

    public void orchestrateUserCreation(UserEvent event) {
        // 1. Anomaly Detection Agent
        AgentDecision anomalyDecision = anomalyAgent.decide(event);
        anomalyAgent.act(anomalyDecision);

        if (anomalyDecision.getAction() == AgentAction.QUARANTINE) {
            // 2. Auto-create Jira ticket
            jiraAgent.decide(anomalyDecision);
            jiraAgent.act(jiraDecision);
            return; // Stop processing
        }

        // 3. Continue normal flow
        processNormally(event);
    }

    @Scheduled(fixedRate = 30000) // Every 30 seconds
    public void monitorHealth() {
        HealthStatus status = healthService.checkAllServices();

        // Self-healing agent monitors continuously
        AgentDecision decision = selfHealingAgent.decide(status);
        selfHealingAgent.act(decision);
    }
}
```

4. Learning & Improvement:

```
@Override
public void learn(AgentFeedback feedback) {
    if (feedback.isCorrect()) {
        // Reinforce behavior
        adjustThresholds(feedback.getDecision(), 0.95); // More lenient
        correctDecisions++;
    } else {
        // Adjust behavior
        adjustThresholds(feedback.getDecision(), 1.05); // More strict
        incorrectDecisions++;
    }
}
```

```
double accuracy = (double) correctDecisions / (correctDecisions +
incorrectDecisions);
logger.info("Agent accuracy: {}%", accuracy * 100);
}
```

Benefits of Agentic AI:

- **Autonomous:** No human intervention for routine tasks
- **24/7 Operation:** Never sleeps, always monitoring
- **Fast Response:** Milliseconds vs hours
- **Consistent:** No human error or bias
- **Learning:** Improves over time with feedback
- **Cost Savings:** Reduces manual labor by 70%

Technical Decisions:

- Used **rule-based AI** (not ML) for predictability
- Each agent is stateless for scalability
- Agents run async (no blocking)
- Comprehensive logging for audit trail
- Feature flags to disable agents if needed

Future Enhancements:

- Integrate GPT-4 for natural language reasoning
- Multi-agent collaboration (agents discuss decisions)
- Predictive maintenance (prevent issues before they occur)
- Automated A/B testing of agent strategies"

□ Resilience & Fault Tolerance

Q21: "How did you implement resilience patterns with Resilience4j?"

Answer: "I implemented comprehensive fault tolerance using **Resilience4j** with Circuit Breakers, Rate Limiters, Retry, and Bulkhead patterns:

1. Circuit Breaker Pattern:

```
@Configuration
public class Resilience4jConfig {

    @Bean
    public CircuitBreakerConfig circuitBreakerConfig() {
        return CircuitBreakerConfig.custom()
            .failureRateThreshold(50) // Open if 50% fail
            .waitDurationInOpenState(Duration.ofSeconds(30)) // Wait 30s before
retry
            .slidingWindowSize(10) // Last 10 calls
```

```

        .permittedNumberOfCallsInHalfOpenState(3)    // Test with 3 calls
        .automaticTransitionFromOpenToHalfOpenEnabled(true)
        .build();
    }
}

```

Usage:

```

@Service
public class ExternalUserService {

    @CircuitBreaker(name = "externalUserService", fallbackMethod =
"fallbackGetUser")
    @Retry(name = "externalUserService", fallbackMethod = "fallbackGetUser")
    @RateLimiter(name = "externalUserService")
    public User getExternalUser(Long userId) {
        // Call to external API (can fail)
        return restTemplate.getForObject(
            "https://api.external.com/users/" + userId,
            User.class
        );
    }

    // Fallback method - same signature + Exception param
    public User fallbackGetUser(Long userId, Exception ex) {
        logger.warn("Circuit breaker fallback for user {}: {}", userId,
ex.getMessage());

        // Return cached data or default
        return userCacheService.getCachedUser(userId)
            .orElse(User.createDefault(userId));
    }
}

```

Circuit Breaker States:

```

CLOSED (normal)  50% failures  OPEN (reject calls immediately)
  HALF_OPEN (test with 3 calls)
    Success  CLOSED
    Failure  OPEN

```

2. Retry Pattern with Exponential Backoff:

```

resilience4j:
  retry:
    instances:
      externalUserService:

```

```

max-attempts: 3
wait-duration: 1s
exponential-backoff-multiplier: 2
retry-exceptions:
  - java.net.SocketTimeoutException
  - org.springframework.web.client.ResourceAccessException

```

Behavior:

- Attempt 1: Immediate
- Attempt 2: Wait 1s
- Attempt 3: Wait 2s
- Attempt 4: Wait 4s (fail)

3. Rate Limiter:

```

resilience4j:
  ratelimiter:
    instances:
      externalUserService:
        limit-for-period: 10      # 10 calls
        limit-refresh-period: 1s  # Per second
        timeout-duration: 0s     # Don't wait, fail immediately

```

4. Bulkhead Pattern (Thread Isolation):

```

resilience4j:
  bulkhead:
    instances:
      externalUserService:
        max-concurrent-calls: 10  # Max 10 concurrent calls
        max-wait-duration: 0ms    # Don't queue requests

```

Purpose: Isolate failures - if external API is slow, only 10 threads blocked, not entire app.

5. Time Limiter (Timeout):

```

@TimeLimiter(name = "externalUserService", fallbackMethod = "fallbackGetUser")
public CompletableFuture<User> getExternalUserAsync(Long userId) {
    return CompletableFuture.supplyAsync(() ->
        restTemplate.getForObject(
            "https://api.external.com/users/" + userId,
            User.class
        )
    );
}

```

```

resilience4j:
  timelimiter:
    instances:
      externalUserService:
        timeout-duration: 5s          # Kill after 5 seconds

```

6. Combining Multiple Patterns:

```

@CircuitBreaker(name = "payment")
@Retry(name = "payment")
@RateLimiter(name = "payment")
@Bulkhead(name = "payment")
@TimeLimiter(name = "payment")
public CompletableFuture<PaymentResponse> processPayment(PaymentRequest request) {
    return paymentGateway.charge(request);
}

```

Execution Order:

1. Bulkhead (check thread availability)
2. TimeLimiter (start timeout)
3. CircuitBreaker (check if open)
4. RateLimiter (check rate limit)
5. Retry (on failure)
6. Fallback (if all retries fail)

7. Monitoring & Metrics:

```

@Component
public class CircuitBreakerEventListener {

    @EventListener
    public void onCircuitBreakerEvent(CircuitBreakerOnStateTransitionEvent event)
    {
        logger.warn("Circuit breaker {} transitioned from {} to {}",
            event.getCircuitBreakerName(),
            event.getStateTransition().getFromState(),
            event.getStateTransition().getToState()
        );

        // Send alert if opened
        if (event.getStateTransition().getToState() == CircuitBreaker.State.OPEN)
        {
            alertService.sendAlert("Circuit breaker opened: " +
            event.getCircuitBreakerName());
        }
    }
}

```

Actuator Endpoints:

```
GET /actuator/circuitbreakers
GET /actuator/circuitbreakerevents
GET /actuator/ratelimiters
GET /actuator/retries
```

Real Incident - External API Outage:

- **Without Resilience:** All threads blocked waiting, entire system down
- **With Resilience4j:**
 - Circuit breaker opened after 50% failures (10 requests)
 - Subsequent requests failed fast (1ms vs 30s timeout)
 - Fallback to cached data
 - System remained operational
 - Automatic recovery when API came back

Metrics Collected:

- Circuit breaker state changes: 5 OPEN events in 1 hour
- Fallback success rate: 98% (cached data available)
- Response time: 30s 1ms (during outage)
- User impact: 2% (degraded) vs 100% (without resilience)

Best Practices:

- Use circuit breakers for external services
- Always provide fallback methods
- Set aggressive timeouts (fail fast)
- Monitor circuit breaker state changes
- Test failure scenarios (chaos engineering)
- Combine multiple patterns for defense in depth
- Use bulkhead to isolate failures
- Implement graceful degradation

Testing:

```
@Test
public void testCircuitBreakerOpens() {
    // Simulate 10 failures
    for (int i = 0; i < 10; i++) {
        assertThrows(CallNotPermittedException.class, () ->
            service.getExternalUser(123L)
        );
    }
}
```



```
// Circuit breaker should be OPEN
CircuitBreaker cb =
circuitBreakerRegistry.circuitBreaker("externalUserService");
assertEquals(CircuitBreaker.State.OPEN, cb.getState());
}
```

```

---

## ## ✕ STAR Format Interview Answers

### Example 1: "Tell me about implementing microservices from monolith"

### \*\*Situation:\*\*

"Our monolithic Spring Boot application had 50 controllers, 200 services, single database. Deployment took 30 minutes, any bug required full redeploy."

### \*\*Task:\*\*

"Break into 5 microservices (Evaluation, Sampling, Evidence, Remediation, JIRA), deploy to AWS with zero downtime."

### \*\*Action:\*\*

1. Identified bounded contexts using Domain-Driven Design
2. Created common module for shared code (security, DTOs, configs)
3. Extracted services one by one (Strangler Fig pattern)
4. Implemented API Gateway pattern with ALB
5. Used Kafka for async communication
6. Deployed to ECS Fargate with Terraform
7. Migrated data with Flyway migrations
8. Implemented distributed tracing (X-Ray)
9. Added circuit breakers (Resilience4j)
10. Tested thoroughly with integration tests"

### \*\*Result:\*\*

- Deployment time: 30 min 5 min per service
- Independent scaling (eval service 10 instances, others 2)
- Fault isolation (one service down, others work)
- Team velocity: 2 releases/month 20 releases/month
- Infrastructure cost: -40% (Fargate auto-scaling)
- Zero downtime deployments with blue-green"

---

## ## ~ Quick Fire Questions

**\*\*Q: What's the difference between @Async and CompletableFuture?\*\***

A: @Async is annotation-driven, Spring manages threads. CompletableFuture is code-driven, you control executors. I use CompletableFuture for fine-grained control (separate I/O and CPU pools).

**\*\*Q: How do you test JWT authentication?\*\***

A: MockMvc with @WithMockUser, integration tests with TestRestTemplate, Postman for manual testing, JMeter for load testing.

**\*\*Q: What's the cost of your AWS infrastructure?\*\***

A: Dev: ~\$80/month (ECS Fargate \$15, RDS \$15, NAT Gateway \$30, ALB \$20).  
Production would be ~\$300 with multi-AZ RDS, larger instances.

**\*\*Q: How do you monitor microservices?\*\***

A: CloudWatch for logs/metrics, X-Ray for distributed tracing, Prometheus + Grafana for custom metrics, ELK stack for log aggregation, Spring Boot Actuator for health checks.

**\*\*Q: What's your Git workflow?\*\***

A: GitFlow: main (production), develop (staging), feature branches. Pull requests required, CI/CD on merge, semantic versioning (1.2.3).

---

**\*\*This comprehensive guide covers 3+ years of experience with Spring Boot, JWT, OAuth2, AWS, Terraform, and microservices!\*\***